

*Communauté Économique et Monétaire de
l'Afrique Centrale
(CEMAC)*



*Institut Sous-régional de Statistique
et d'Économie Appliquée*

Organisation Internationale

BP : 294 Yaoundé - Tél : 222 220 134

Cameroun AI Enthousiaste



*Direction du Développement et de la
Recherche*

ONG camerounaise

BP : 83000 Toulon, France

*Mémoire professionnel de fin d'étude présenté en vue de l'obtention du diplôme d'Ingénieur
Statisticien Économiste(ISE)*

OPTION : Data Science et Marketing

**CONCEPTION D'UN SYSTEME DE RECOMMANDATION HYBRIDE POUR LE
MATCHING EMPLOI-COMPETENCES AU CAMEROUN PAR INTEGRATION DES
LLMS ET DES GRAPHS DE CONNAISSANCES**

Rédigé par :

NGOULOU NGOUBILI Irch Defluviaire

Élève Ingénieur Statisticien Économiste cycle long – 5^e année

Sous l'encadrement de :

Dr. FOUTSE Yuehgoh

Présidente & Directrice exécutive adjointe, PhD Computer Science & Data Scientist

Année académique 2025-2026

Dédicace

À mes parents... (Max 1 page)

Remerciements

Sommaire

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	v
Liste des tableaux	vi
Liste des figures	vii
Avant-propos	viii
Résumé	ix
Abstract	x
Introduction générale	1
I Cadre théorique et conceptuel	6
1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation	7
1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG	7
1.2 Fondements économiques et problématique d'appariement	9
1.3 Systèmes de recommandation et person-job fit	10
1.4 LLMs, embeddings et graphes de connaissances	12
1.5 RAG, GraphRAG et orchestration agentique	14
1.6 Positionnement de l'étude	18
2 Sources de données et approche méthodologique	19
2.1 Cadre méthodologique et sources de données	19
2.2 Représentation sémantique et graphe de connaissances	21

2.3	Orchestration agentique et génération contrôlée	24
II	Présentation des résultats	25
3	Construction opérationnelle du système hybride emploi-compétences	26
3.1	Données exploitables et contraintes de qualité du système	26
3.2	Adaptation du modèle d’embeddings au domaine emploi-compétences	28
3.3	Mémoire vectorielle pgvector pour le retrieval	33
3.4	Neo4j : mémoire relationnelle du matching	35
3.5	Score hybride et reranking des recommandations	39
3.6	Orchestration du système par LangGraph	40
4	Évaluation de la performance du système	43
4.1	Protocole expérimental et métriques retenues	43
4.2	Retrieval sémantique et apport mesuré du graphe	44
4.3	Validation fonctionnelle du GraphRAG	46
4.4	Reranking hybride et calibration du score	49
4.5	Contrôle de génération par critic	52
	Conclusion générale	55
	Bibliographie	II
A	Annexes	VI

Liste des sigles et abréviations

ANN :	Approximate Nearest Neighbors
API :	Application Programming Interface
BERT :	Bidirectional Encoder Representations from Transformers
BI :	Business Intelligence
CEMAC :	Communauté Économique et Monétaire de l'Afrique Centrale
CV :	Curriculum Vitae
DCG :	Discounted Cumulative Gain
ESCO :	European Skills, Competences, Qualifications and Occupations
FAISS :	Facebook AI Similarity Search
FNE :	Fonds National de l'Emploi
FRED :	Federal Reserve Economic Data
GAT :	Graph Attention Network
GCN :	Graph Convolutional Network
GNN :	Graph Neural Network
GPT :	Generative Pre-trained Transformer
GPU :	Graphics Processing Unit
HNSW :	Hierarchical Navigable Small World
IA :	Intelligence Artificielle
IDCG :	Ideal Discounted Cumulative Gain
ISE :	Ingénieur Statisticien Économiste
ISSEA :	Institut Sous-régional de Statistique et d'Économie Appliquée
IVF :	Inverted File Index
KG :	Knowledge Graph
LLM :	Large Language Model
LoRA :	Low-Rank Adaptation
LSTM :	Long Short-Term Memory
MAP :	Mean Average Precision
MEPC :	Nomenclature Camerounaise des Métiers, Emplois et Professions
MRR :	Mean Reciprocal Rank
NCF :	Nomenclature Camerounaise des Formations
NCM :	Nomenclature Camerounaise des Métiers
NDCG :	Normalized Discounted Cumulative Gain
NLP :	Natural Language Processing
OIT :	Organisation Internationale du Travail
ONG :	Organisation Non Gouvernementale
QLoRA :	Quantized Low-Rank Adaptation
RAG :	Retrieval-Augmented Generation
RNN :	Recurrent Neural Network
RWKV :	Receptance Weighted Key Value
SEO :	Search Engine Optimization
SGPT :	Sentence Generative Pre-trained Transformer
SQL :	Structured Query Language
SR :	Système de Recommandation

Liste des tableaux

Tableau 2.1	Sources de données mobilisées et utilisation méthodologique	20
Tableau 3.1	Protocole de construction des paires de fine-tuning	29
Tableau 3.2	Hyperparamètres du fine-tuning SentenceTransformer	30
Tableau 3.3	Distribution des embeddings par type d'entité	34
Tableau 3.4	Résultats de calibration des poids du score hybride	40
Tableau 4.1	Benchmark des modèles d'embeddings sur 473 requêtes de test	44
Tableau 4.2	Comparaison retrieval : pgvector seul versus pgvector + graphe	45
Tableau 4.3	Pondérations retenues selon la fonction objectif du score hybride . . .	50
Tableau A.1	Labels de nœuds et relations structurantes du graphe	VIII
Tableau A.2	Volume d'entités indexées dans la base vectorielle pgvector	IX
Tableau A.3	Pipeline général implémenté : étapes, entrées et sorties	IX
Tableau A.4	Correspondance entre objectifs spécifiques et choix méthodologiques	X
Tableau A.5	Volumes réellement exploités par les modules de recommandation . .	XI
Tableau A.6	Synthèse des traitements de nettoyage utiles au moteur de recomman- dation	XII
Tableau A.7	Comparaison directe entre le baseline et le modèle fine-tuné	XIII
Tableau A.8	Diagnostic statistique du chunking documentaire	XIII
Tableau A.9	Couverture documentaire et duplication des chunks	XIV
Tableau A.10	Cohérence sémantique intra-chunk	XIV
Tableau A.11	Hétérogénéité des degrés par famille de noeuds	XV
Tableau A.12	Résumé du déplacement de rang après reranking Neo4j	XVI

Table des figures

Figure 1.1	Évolution simplifiée des modèles de langage et des représentations textuelles	12
Figure 1.2	Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG	15
Figure 3.1	Répartition des offres par source après nettoyage	26
Figure 3.2	Compétences fréquentes et disponibilité descriptive	27
Figure 3.3	Diagnostic du fine-tuning : perte, généralisation, métriques de validation et calendrier du learning rate	31
Figure 3.4	Comparaison des modèles d’embeddings : qualité de classement, rappel et latence	32
Figure 3.5	Projection UMAP des embeddings indexés	32
Figure 3.6	Schéma physique de la base pgvector	33
Figure 3.7	Pipeline de chunking structurel des référentiels documentaires	34
Figure 3.8	Distribution des tailles, sources et couverture du chunking documentaire	35
Figure 3.9	Schéma logique du graphe de connaissances	36
Figure 3.10	Lecture conjointe des noeuds et relations Neo4j mobilisés par le matching	36
Figure 3.11	Tableau de bord statistique du graphe Neo4j	37
Figure 3.12	Distribution des degrés dans la projection matching	38
Figure 3.13	Distribution comparée des scores pgvector et Neo4j utilisés dans le reranking	39
Figure 3.14	Workflow agentique orchestré dans LangGraph Studio	41
Figure 4.1	Comparaison des modèles d’embeddings sur le jeu de test	45
Figure 4.2	Effet de l’enrichissement graphe sur les métriques de retrieval	46
Figure 4.3	Évaluation fonctionnelle des requêtes Neo4j utiles au GraphRAG	47
Figure 4.4	Distribution des déplacements de rang après reranking par le graphe	49
Figure 4.5	Évolution de l’objectif NDCG@10 au cours des essais Optuna	51
Figure 4.6	Comparaison des pondérations retenues selon la fonction objectif	51
Figure 4.7	Distribution des scores du critic avec contexte réel et contexte mélangé	52
Figure 4.8	Effet du seuil du critic sur les décisions accept/revise	53
Figure A.1	Architecture méthodologique du workflow agentique de recommandation	VI
Figure A.2	Interface Streamlit du chatbot Agentic GraphRAG	VII
Figure A.3	Localisation nettoyée des offres	XII
Figure A.4	Distribution de la cohérence sémantique intra-chunk par source	XIV
Figure A.5	Composition du graphe par types de noeuds et relations	XV

Avant-propos

L'Institut Sous-régional de Statistique et d'Économie Appliquée (ISSEA), institution spécialisée de la CEMAC créée en 1984, a pour mission principale la formation de cadres supérieurs en statistique, économie et data science, capables de répondre aux enjeux contemporains de décision fondée sur les données. Dans ce cadre, l'ISSEA propose plusieurs filières de formation, parmi lesquelles figure celle des Ingénieurs Statisticiens Économistes (ISE), alliant rigueur mathématique, modélisation économétrique et outils modernes d'analyse de données (Data Science).

Arrivés à un niveau avancé de leur formation, les élèves Ingénieurs Statisticiens Économistes sont tenus d'effectuer un stage professionnel d'une durée minimale de trois (03) mois. Ce stage constitue une phase essentielle du cursus, permettant de confronter les connaissances théoriques acquises à des problématiques réelles, tout en favorisant l'acquisition de compétences opérationnelles en environnement professionnel. Il est sanctionné par la rédaction d'un mémoire professionnel, soutenu devant un jury.

C'est dans ce cadre que nous avons effectué un stage du 9 mars 2026 au 30 juin 2026 au sein de KmerAI, où nous avons été intégrés à la Direction du Développement et de la Recherche. Cette immersion nous a permis de travailler sur une problématique appliquée située au croisement de l'analyse de données, de l'intelligence artificielle générative, du traitement automatique du langage naturel et de l'aide à la décision : l'amélioration du matching entre offres d'emploi, profils de candidats et compétences recherchées sur le marché camerounais.

Le présent mémoire s'inscrit dans cette dynamique et porte sur le thème : « Conception d'un système de recommandation hybride pour le matching emploi-compétences au Cameroun par intégration des LLMs et des graphes de connaissances ». Il vise principalement à concevoir une architecture capable de structurer les données d'offres et de profils, d'exploiter les référentiels métiers et formations, puis de produire des recommandations plus explicables que les approches fondées uniquement sur des mots-clés. Plus spécifiquement, il s'agit de normaliser les données disponibles, d'aligner les métiers et compétences avec les référentiels ESCO, MEPC et NCF, d'adapter un modèle d'embeddings au domaine emploi-compétences, de construire un graphe de connaissances sous Neo4j et de poser les bases d'un moteur de recommandation hybride intégrant similarité sémantique, relations de graphe et analyse du *skill gap*.

Ce travail répond à un double enjeu. Sur le plan scientifique et méthodologique, il cherche à montrer comment les modèles de représentation sémantique, les bases vectorielles et les graphes de connaissances peuvent être combinés pour traiter un problème réel d'appariement sur le marché du travail. Sur le plan opérationnel, il propose une démarche reproductible susceptible d'aider KmerAI à développer des outils de recommandation, d'orientation et de montée en compétences adaptés aux besoins des candidats, des recruteurs et des acteurs de la formation.

Résumé

Mots-clés : Économétrie, Statistique, Développement...

Abstract

INTRODUCTION GÉNÉRALE

Contexte et justification

Le fonctionnement du marché de l'emploi constitue un déterminant central de la performance économique et sociale d'un pays. Au Cameroun, ce marché reste marqué par des déséquilibres structurels persistants, notamment l'inadéquation entre les compétences disponibles et les besoins des entreprises, la forte informalité et les difficultés d'insertion professionnelle des jeunes. Les données disponibles montrent que le chômage des jeunes demeure une réalité importante, avec un taux estimé à 6,6% en 2024 et 6,5% en 2025 selon les estimations modélisées de l'OIT relayées par la Banque Mondiale et la base FRED. Par ailleurs, plusieurs travaux sur le Cameroun soulignent que le problème ne tient pas uniquement au manque d'emplois, mais aussi à un mismatch entre formation, compétences et exigences du marché.

Dans ce contexte, la question du matching emploi-compétences devient centrale. En théorie, un marché du travail efficient devrait permettre d'associer rapidement les profils disposant des compétences requises aux postes vacants correspondants. En pratique, plusieurs frictions informationnelles persistent : asymétrie d'information entre recruteurs et candidats, faible structuration des profils de compétences, hétérogénéité des offres d'emploi et faiblesse des systèmes d'intermédiation comme le FNE et certains agences de recrutement. Ces dysfonctionnements conduisent à une situation paradoxale où coexistent des postes non pourvus et des chercheurs d'emploi, traduisant une inefficacité allocative du marché du travail.

Par ailleurs, l'essor des technologies numériques et de l'intelligence artificielle ouvre des perspectives nouvelles pour améliorer l'appariement entre offres et profils. Dans cette dynamique, KmerAI se positionne comme une plateforme camerounaise dédiée à la promotion de l'intelligence artificielle, à la vulgarisation de ses usages et à l'accompagnement des étudiants, chercheurs et entreprises dans l'adoption de solutions innovantes adaptées au contexte local. Ses missions consistent notamment à former aux bases de l'IA, à offrir un espace de partage d'expériences et à encourager la collaboration autour de cas d'usage appliqués à des secteurs stratégiques tels que la santé, l'agriculture, le transport, l'éducation et le marketing.

Cette orientation est particulièrement pertinente pour le marché de l'emploi camerounais, où les données liées aux offres, aux CV et aux compétences restent souvent dispersées et peu exploitées. Une approche fondée sur l'intelligence artificielle peut permettre d'analyser plus efficacement ces données hétérogènes afin de proposer des recommandations d'emploi plus

pertinentes, plus rapides et plus personnalisées.

Problématique

Le marché de l'emploi camerounais est marqué par une forte asymétrie d'information, une hétérogénéité des profils professionnels et des offres d'emploi, ainsi qu'une faible capacité des outils classiques à évaluer les écarts réels de compétences. Les systèmes fondés sur des mots-clés ou des filtres statiques ne parviennent pas à détecter les compétences implicites, les équivalences sémantiques et les trajectoires de montée en compétences.

Par ailleurs, si des référentiels structurants existent notamment la Nomenclature Camerounaise des Métiers (NCM 2013), le référentiel européen ESCO et la Nomenclature Camerounaise des Formations (NCF), leur intégration opérationnelle dans les plateformes et outils d'appariement reste encore limitée. Ils sont peu exploités pour aligner les intitulés locaux de métiers sur des standards internationaux, harmoniser les compétences attendues, qualifier les niveaux et domaines de formation, et rendre les correspondances profil-offre plus transparentes et comparables. Il en résulte un décalage entre la richesse potentielle de ces référentiels et leur usage effectif dans les systèmes d'information dédiés à l'emploi et à l'orientation.

Dans ce contexte, la question centrale n'est plus seulement d'apparier un profil à une offre, mais de concevoir un système capable (i) d'exploiter conjointement les référentiels NCM, ESCO et NCF pour structurer les métiers, les compétences et les formations, (ii) de mesurer finement le *skill gap* entre un candidat et un poste cible, et (iii) de proposer un parcours de montée en compétences compatible avec le niveau, le domaine de formation et les trajectoires de qualification du candidat. La question principale de recherche peut ainsi être formulée comme suit :

Comment concevoir un système de recommandation hybride, fondé sur les LLMs, les graphes de connaissances, les embeddings vectoriels et le filtrage collaboratif, capable de mesurer le skill gap entre un profil et une offre d'emploi, puis de recommander un parcours de montée en compétences cohérent avec la nomenclature des formations, le niveau du candidat et le métier visé pour les membres de la communauté KmerAi ?

Cette question générale se décline en plusieurs questions de recherche sous-jacentes :

- **Q1** : Comment identifier et formaliser, dans le contexte camerounais, les compétences, métiers, formations et relations pertinentes pour un matching emploi compétences fiable et opérationnel à partir des référentiels NCM, ESCO et NCF ?

- **Q2** : Dans quelle mesure l'intégration d'un graphe de connaissances améliore-t-elle la qualité et la précision du matching par rapport aux approches purement vectorielles ?

Objectifs de l'étude

Objectif général. L'objectif général de cette étude est de concevoir et d'évaluer un système de recommandation hybride emploi-compétences pour appuyer la décision de recrutement et la montée en compétence. Le système rapproche les profils candidats et les offres d'emploi en combinant recherche sémantique, référentiels métiers et formations, graphe de connaissances et analyse explicable du *skill gap*. Il vise ainsi à identifier les offres accessibles, les écarts de compétences, les profils à conserver en vivier et les compétences à développer.

Objectifs spécifiques. Plus spécifiquement, il s'agit de :

- normaliser les offres et profils, puis les aligner avec les référentiels ESCO, MEPC et NCF pour obtenir des variables de matching fiables ;
- adapter un modèle *SentenceTransformer* au domaine emploi-compétences et stocker ses embeddings dans *pgvector* ;
- construire un graphe Neo4j des relations candidats-offres-compétences et définir un score hybride de recommandation ;
- orchestrer le pipeline avec *LangGraph*, produire des verdicts interprétables et évaluer le retrieval par Recall@K, NDCG@K, MRR@K et MAP@K.

Intérêt de l'étude

L'intérêt d'une telle approche est renforcé par les limites des méthodes classiques de recrutement, souvent fondées sur des critères statiques ou sur des mots-clés, qui ne captent pas toujours la richesse des parcours et des compétences. En combinant différentes logiques de recommandation, un système hybride peut mieux prendre en compte les dimensions explicites et implicites des profils candidats, tout en améliorant la qualité des correspondances proposées. Cela rejoint la mission de KmerAI, qui vise à favoriser l'appropriation locale de l'IA pour résoudre des problèmes concrets du développement camerounais.

Hypothèses de recherche

Afin de répondre à la problématique, cette étude repose sur les hypothèses suivantes.

- **H1** : L'intégration conjointe de la NCM, d'ESCO et de la NCF dans un référentiel unifié métiers-compétences-formations améliore la structuration et les performances d'appariement profil-offre, par rapport à une approche sans ces référentiels ;

- **H2** : La modélisation du marché de l’emploi sous forme de graphe de connaissances (Neo4j), incluant explicitement les niveaux et domaines de formation de la NCF, améliore la qualité du matching par rapport à une approche purement vectorielle ;
- **H3** : Le *fine-tuning* d’un modèle de représentation sémantique sur le domaine emploi-compétences améliore la qualité des embeddings et la similarité sémantique entre profils et offres, par rapport au même modèle non adapté ;
- **H4** : La combinaison d’un RAG vectoriel, d’un GraphRAG et d’un score de recommandation hybride améliore la pertinence et l’explicabilité des recommandations de *skill gap* et de parcours de montée en compétences, notamment lorsque ces parcours sont contraints par la nomenclature des formations.

Méthodologie de recherche

La démarche adoptée s’inscrit dans un cadre d’ingénierie des données combinant structuration des connaissances et modélisation hybride. Elle repose sur la collecte et la normalisation des données issues d’offres d’emploi sur les différentes sites web et de profils, alignées sur la NCM 2013 afin d’assurer la cohérence sémantique. Un graphe de connaissances est ensuite construit sous Neo4j en intégrant les référentiels ESCO et NCM, à partir d’un processus d’extraction automatique de triplets à partir des données textuelles. Parallèlement, les descriptions sont vectorisées et indexées dans un espace sémantique permettant la mesure de similarité. Le moteur de recommandation combine ces différentes sources d’information à travers un score hybride articulant similarité vectorielle et signal relationnel issu du graphe. Le système permet ainsi d’identifier les écarts de compétences entre profils et offres, et de générer des recommandations sous forme de trajectoires d’apprentissage. L’évaluation repose sur des métriques standard telles que la précision@K, le rappel et le NDCG, ainsi que sur l’analyse de la cohérence des recommandations générées.

La réalisation du système mobilise Python pour l’ETL, l’analyse et l’entraînement ; SentenceTransformers et PyTorch pour les embeddings ; PostgreSQL avec pgvector pour la mémoire vectorielle ; Neo4j pour la mémoire relationnelle ; LangGraph/LangChain pour l’orchestration agentique ; Optuna pour la calibration du score hybride ; FastAPI et Streamlit pour l’exposition applicative. Les traitements ont été exécutés sur un ordinateur local de type AMD64 disposant de 8 processeurs logiques et d’environ 15,25 Go de mémoire vive. PyTorch y est disponible en mode CPU, sans GPU CUDA détecté. Cette contrainte matérielle justifie les choix d’implémentation retenus : modèle d’embeddings léger, vecteurs de dimension 384, lots d’entraînement contrôlés et recours à une API externe pour la génération finale.

Plan du travail

Le mémoire est structuré en trois parties. La première présente le cadre théorique relatif aux systèmes de recommandation, aux modèles de langage et aux graphes de connaissances. La deuxième détaille l'approche méthodologique, incluant la modélisation du graphe et la conception du système hybride. La troisième analyse les résultats obtenus et discute les performances du modèle ainsi que ses implications dans le contexte du marché du travail camerounais.

Première partie

Cadre théorique et conceptuel

Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation

Ce chapitre situe le mémoire dans la littérature sur l'appariement emploi-compétences, les systèmes de recommandation, l'intelligence artificielle générative, les graphes de connaissances et la génération augmentée par récupération. Il ne vise pas à reprendre l'ensemble des raisonnements des ouvrages ou articles consultés. Il sélectionne les résultats qui éclairent directement le thème du stage : concevoir un système capable de recommander des offres, d'identifier des écarts de compétences et de produire des explications fondées sur des données structurées et non structurées.

La revue est guidée par une hypothèse centrale : le matching emploi-compétences n'est pas un simple problème de similarité textuelle. Il combine des frictions du marché du travail, une forte variabilité du vocabulaire, des contraintes de qualification, des relations métier-compétence-formation et un besoin d'explicabilité. Cette nature composite explique pourquoi la littérature pertinente relève à la fois de l'économie du travail, de la recommandation hybride, de la recherche sémantique, des graphes de connaissances et de la génération contrôlée.

1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG

Avant d'entrer dans la revue de littérature, il est nécessaire de stabiliser les concepts utilisés dans la suite du mémoire. Cette clarification évite une confusion fréquente : employer indifféremment IA générative, LLM, RAG, GraphRAG, moteur de recommandation et chatbot. Dans ce travail, ces termes désignent des fonctions complémentaires, mais non interchangeables.

Matching emploi-compétences.

Le matching emploi-compétences désigne le processus qui consiste à rapprocher un profil candidat d'une offre, d'un métier ou d'une trajectoire de formation à partir de caractéristiques observables : compétences, niveau de formation, expérience, secteur, localisation et objectif professionnel. Il est traité ici comme un problème d'appariement économique, mais aussi comme un problème de représentation des connaissances.

Système de recommandation.

Un système de recommandation est un dispositif algorithmique qui classe des éléments selon leur pertinence probable pour un utilisateur ou un cas d'usage [1]. Dans ce mémoire, les éléments recommandés peuvent être des offres d'emploi, des compétences, des métiers ou des pistes de formation. La pertinence ne renvoie donc pas seulement à une préférence ; elle renvoie aussi à une compatibilité professionnelle explicable.

Skill gap.

Le *skill gap*, ou écart de compétences, correspond à la différence entre les compétences déjà détenues par un candidat et celles attendues pour une offre ou un métier cible. Il permet de dépasser une logique binaire compatible/non compatible : un candidat peut être proche d'un poste tout en nécessitant une montée en compétences.

Embedding et recherche sémantique.

Un embedding est une représentation vectorielle d'un texte, d'un profil, d'une offre ou d'une compétence. La recherche sémantique utilise cette représentation pour retrouver des objets proches en sens, même lorsque les formulations diffèrent. Elle répond à un problème récurrent des données d'emploi : des expressions comme « data analyst », « analyste de données » et « reporting statistique » peuvent désigner des zones de compétences voisines.

Base vectorielle pgvector.

pgvector désigne l'extension PostgreSQL utilisée pour stocker et rechercher des embeddings [2]. Dans ce mémoire, elle sert de couche de recherche rapide pour retrouver les offres, métiers, compétences ou documents proches d'une requête. Son rôle est de présélectionner des candidats à la recommandation. Elle ne suffit cependant pas à décider seule, car un bon score vectoriel ne garantit pas que le niveau NCF, l'expérience ou les compétences obligatoires soient compatibles.

Graphe de connaissances Neo4j.

Un graphe de connaissances représente des entités et leurs relations : candidat, offre, compétence, métier, secteur, niveau, formation et référentiel [3]. Dans ce mémoire, Neo4j sert à représenter les relations explicites entre ces entités. Il permet de répondre à des questions que la seule similarité vectorielle ne traite pas correctement : quelles compétences une offre exige-t-elle ? quelles compétences le candidat possède-t-il ? quel niveau de formation est demandé ? quels chemins relient un métier à une formation ?

IA générative, LLM et prompting.

L'intelligence artificielle générative désigne les modèles capables de produire du contenu, notamment du texte. Un LLM est un grand modèle de langage entraîné sur de grands corpus textuels et capable de générer, reformuler ou synthétiser des réponses. Dans ce mémoire, le LLM n'est pas le moteur de vérité du système. Il intervient comme générateur contrôlé : il reçoit un contexte construit à partir de pgvector, Neo4j et des documents, puis rédige une réponse en français. Le prompting désigne l'ensemble des consignes données au modèle pour encadrer son comportement, limiter les inventions et structurer la réponse.

RAG, GraphRAG et Agentic RAG.

La RAG combine récupération d'information et génération : le système recherche d'abord des éléments pertinents, puis les transmet au LLM pour produire une réponse [4]. Le GraphRAG étend cette logique en ajoutant des faits et chemins issus d'un graphe de connaissances. L'Agentic RAG ajoute une couche d'orchestration : l'agent analyse l'intention, choisit les outils, récupère les données, construit le contexte, génère la réponse et peut demander une révision si le contexte est insuffisant. Dans ce mémoire, cette orchestration est assurée par LangGraph.

Critic et traçabilité.

Le critic est un mécanisme de contrôle qui examine si la réponse générée semble suffisamment couverte par le contexte récupéré. Il ne remplace pas une validation humaine, mais il réduit le risque d'une réponse fluide et insuffisamment fondée. La traçabilité désigne la capacité à afficher les outils mobilisés, les sources consultées et le chemin suivi par le workflow. Elle est indispensable dans un système d'aide à la décision, car une recommandation professionnelle doit être vérifiable.

1.2 Fondements économiques et problématique d'appariement

1.2.1 Marché du travail, recherche d'emploi et frictions

Le problème traité dans ce mémoire relève d'abord de l'économie du travail. Les modèles de recherche et d'appariement montrent que le chômage peut coexister avec des postes vacants lorsque la rencontre entre travailleurs et entreprises est coûteuse, lente ou imparfaitement informée [5, 6]. Cette lecture est importante : un système de recommandation emploi-compétences n'est pas seulement un outil informatique ; c'est un dispositif d'intermédiation qui cherche à réduire les coûts de recherche et à améliorer la qualité des mises en relation.

La littérature sur l'asymétrie d'information renforce ce diagnostic. Akerlof montre que l'in-

certitude sur la qualité peut dégrader les échanges lorsque les acteurs ne disposent pas de signaux fiables [7]. Sur le marché de l'emploi, cette asymétrie apparaît des deux côtés : le recruteur ne voit pas toujours les compétences réelles du candidat, et le candidat ne connaît pas toujours les exigences effectives du poste. La théorie du signalement de Spence explique alors le rôle des diplômes, certifications et expériences comme signaux observables [8]. Mais ces signaux restent incomplets lorsque les métiers évoluent rapidement, notamment dans les domaines numériques où les compétences techniques sont souvent décrites par des formulations instables.

1.2.2 Capital humain, tâches et inadéquation des compétences

La théorie du capital humain rappelle que la formation et l'expérience sont des investissements productifs [9]. Toutefois, la possession d'un capital humain ne garantit pas l'appariement. Encore faut-il que les compétences acquises soient visibles, comparables et alignées sur les compétences demandées. Autor montre que le progrès technique transforme surtout les tâches composant les métiers, et non uniquement les intitulés d'emploi [10]. Pour un système de recommandation, cela impose de descendre au niveau des compétences et des tâches plutôt que de s'arrêter aux titres de poste.

L'apport de cette littérature est de montrer que l'inadéquation emploi-compétences relève autant de la circulation imparfaite de l'information que d'un manque de formation. Sa limite, pour notre objet, est qu'elle ne fournit pas directement les outils de traitement des données nécessaires pour extraire, normaliser, comparer et expliquer les compétences. La section suivante examine donc les systèmes de recommandation comme réponse algorithmique à ce problème d'appariement.

1.3 Systèmes de recommandation et person-job fit

1.3.1 Approches par contenu, collaboratives et hybrides

Les systèmes de recommandation cherchent à classer des items selon leur pertinence probable pour un utilisateur. La littérature distingue classiquement les approches fondées sur le contenu, le filtrage collaboratif et les approches hybrides [11, 1]. Dans le contexte de ce mémoire, l'utilisateur peut être un candidat, un conseiller emploi ou une plateforme ; les items peuvent être des offres, compétences, métiers ou formations.

Les approches fondées sur le contenu utilisent les caractéristiques explicites des items et des profils. Elles sont adaptées au matching emploi-compétences parce que les offres et les

candidats possèdent des descriptions textuelles, des secteurs, des niveaux et des compétences. Pazzani et Billsus montrent que ces systèmes exploitent les attributs des contenus pour produire des recommandations personnalisées [12]. Leur avantage est l'interprétabilité ; leur limite est la dépendance au vocabulaire observé.

Le filtrage collaboratif exploite les interactions passées entre utilisateurs et items. Les approches de factorisation matricielle ont fortement structuré ce domaine en apprenant des facteurs latents à partir des matrices d'interactions [13]. Les modèles neuronaux ont ensuite généralisé cette logique, par exemple avec le Neural Collaborative Filtering [14]. Ces méthodes sont puissantes lorsque l'on dispose d'un historique dense : clics, candidatures, recrutements, évaluations ou parcours de formation. Dans le cas étudié ici, cette hypothèse est fragile, car les données d'interaction restent limitées et de nombreux profils se trouvent en situation de démarrage à froid.

Les approches hybrides combinent plusieurs sources de signal pour compenser les limites propres à chaque famille. Burke montre que l'hybridation peut améliorer la robustesse en associant contenu, collaboration, connaissances et règles métier [15]. Cette conclusion prépare directement le choix d'un système hybride : les signaux textuels, relationnels et métier doivent être séparés puis combinés, car chacun ne capture qu'une partie de la compatibilité.

1.3.2 Spécificité du person-job fit

La recommandation d'emploi se distingue de la recommandation de films ou de produits. Une offre n'est pas seulement un item susceptible de plaire ; elle impose des contraintes de qualification, d'expérience, de localisation, de disponibilité et de compétences. La littérature récente sur le *person-job fit* insiste sur cette double sélection : le candidat doit être intéressé par l'offre, mais l'employeur doit aussi pouvoir considérer le profil comme compatible [16]. La recommandation devient donc un problème bilatéral.

Cette spécificité change l'interprétation des scores. Dans le commerce électronique, une recommandation peut viser la probabilité de clic ou d'achat. Dans l'emploi, une recommandation doit distinguer au moins trois situations : le candidat est immédiatement compatible ; le candidat est proche mais nécessite une montée en compétences ; le candidat est trop éloigné du poste cible. La notion de *skill gap* devient alors centrale.

Les travaux récents mobilisant les graphes et les LLM dans les ressources humaines vont dans ce sens. HRGraph propose par exemple d'exploiter des graphes de connaissances enrichis par LLM pour la recommandation d'emploi [17]. D'autres travaux s'intéressent à la prédiction des compétences et à la construction de graphes de talents pour les ressources humaines

[18]. Ces contributions confirment l'intérêt des graphes pour représenter les relations entre compétences, métiers et profils, mais elles ne résolvent pas automatiquement la question de l'ancrage local : nomenclatures camerounaises, données réelles de plateforme, contraintes de formation et explication opérationnelle.

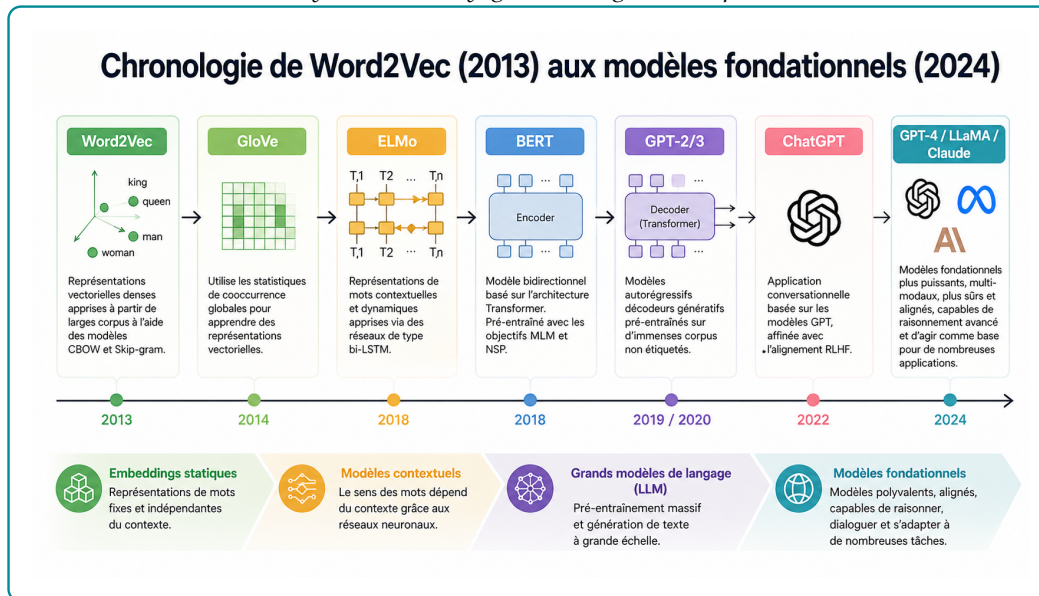
1.4 LLMs, embeddings et graphes de connaissances

1.4.1 Grands modèles de langage et génération contrôlée

Les grands modèles de langage ont transformé le traitement automatique du langage naturel en permettant des tâches de génération, synthèse, reformulation et raisonnement assisté. Leur rupture technique vient principalement de l'architecture Transformer proposée par Vaswani et al., fondée sur le mécanisme d'attention [19]. Avant cette rupture, les représentations textuelles étaient passées des sacs de mots aux embeddings denses comme Word2Vec, qui apprennent des vecteurs capturant des régularités sémantiques et syntaxiques [20]. Les mécanismes d'attention introduits en traduction neuronale ont ensuite permis aux modèles de pondérer dynamiquement les parties pertinentes d'une séquence [21].

Figure 1.1 – Évolution simplifiée des modèles de langage et des représentations textuelles

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



Deux grandes familles de modèles sont utiles pour ce mémoire. La première regroupe les modèles de représentation, comme BERT et Sentence-BERT. BERT apprend des représentations contextuelles bidirectionnelles adaptées à la compréhension du texte [22]. Sentence-BERT adapte cette logique pour produire des embeddings de phrases comparables efficacement par

similarité cosinus [23]. La seconde famille regroupe les modèles génératifs, comme GPT, T5 ou les modèles instruction-tuned. GPT-3 a montré qu'un modèle de grande taille pouvait effectuer des tâches variées à partir de quelques exemples dans le prompt [24]. T5 a proposé une formulation unifiée des tâches NLP comme transformation texte-vers-texte [25]. L'apprentissage par retour humain a ensuite amélioré la capacité des modèles à suivre des instructions [26].

L'acquis de cette littérature est considérable : les LLM savent reformuler, synthétiser et produire des explications lisibles. Sa limite, pour notre problématique, tient au statut de la vérité : un modèle génératif peut produire une réponse plausible sans être fondée sur les données locales. Dans un système d'emploi, cette limite impose une génération contrôlée par des sources récupérées.

1.4.2 Embeddings, recherche sémantique et bases vectorielles

Après la clarification conceptuelle, l'enjeu est de savoir dans quelles conditions les représentations vectorielles sont fiables pour la recherche d'information. Les benchmarks montrent que les modèles d'embeddings doivent être évalués sur des tâches variées, car un bon modèle généraliste peut être insuffisant dans un domaine spécialisé [27, 28]. Cette observation justifie l'adaptation d'un modèle SentenceTransformer au vocabulaire emploi-compétences.

La recherche sémantique résout une partie du problème de vocabulaire, mais elle reste une approximation. Deux textes proches peuvent cacher une incompatibilité : niveau insuffisant, compétence obligatoire absente, localisation incompatible ou métier trop éloigné. La similarité vectorielle doit donc être traitée comme un signal de présélection, non comme une décision finale.

La littérature sur la recherche approximative de plus proches voisins montre par ailleurs que le passage à l'échelle exige des structures d'indexation efficaces. HNSW fournit un cadre robuste pour la recherche vectorielle approximative [29], tandis que FAISS a popularisé la recherche vectorielle à grande échelle [30]. Dans ce mémoire, l'utilisation de PostgreSQL avec pgvector répond à une logique pragmatique : conserver les embeddings dans une base relationnelle exploitable, interrogeable et intégrable au reste du pipeline [2].

1.4.3 Graphes de connaissances et recommandation relationnelle

Hogan et al. montrent que les graphes de connaissances servent à structurer des informations hétérogènes, à rendre explicites des relations sémantiques et à soutenir des tâches de recherche ou de raisonnement [3]. Dans le domaine emploi-compétences, cette représenta-

tion complète la recherche vectorielle : elle rend visibles les relations entre candidats, offres, compétences, métiers, formations et niveaux.

La recommandation par graphe a également connu des avancées importantes. Les graph neural networks ont fourni des outils pour propager de l'information dans des graphes [31, 32]. LightGCN a montré que, dans la recommandation collaborative, une architecture simplifiée de propagation sur graphe pouvait être très efficace [33]. Ces travaux établissent l'intérêt de la structure relationnelle pour la recommandation.

Cependant, le présent mémoire ne cherche pas principalement à entraîner un GNN. Le choix méthodologique est différent : utiliser un graphe de connaissances comme support d'explication, de contraintes et de calcul du skill gap. Ce choix est plus adapté au contexte disponible. Les données d'interaction ne sont pas assez riches pour justifier un modèle collaboratif profond, tandis que les relations métier-compétence-formation sont directement interprétables.

1.5 RAG, GraphRAG et orchestration agentique

1.5.1 De la RAG au GraphRAG

La génération augmentée par récupération, ou RAG, combine une étape de recherche d'information et une étape de génération. Lewis et al. montrent que l'accès à une mémoire externe améliore les tâches intensives en connaissances, car le modèle ne dépend plus uniquement de ses paramètres internes [4]. Les synthèses récentes sur la RAG insistent sur les mêmes enjeux : qualité de la récupération, actualisation des connaissances, évaluation de la factualité et articulation entre retriever et générateur [34].

Dans une RAG classique, le système récupère des documents proches d'une requête, puis les transmet au LLM. Cette architecture est utile, mais elle reste centrée sur des passages textuels. Pour le matching emploi-compétences, le contexte doit aussi intégrer des relations : compétences possédées, compétences requises, niveau de formation, métier cible, secteur et formation possible. Le GraphRAG étend alors la récupération vers des informations structurées par graphe.

Figure 1.2 – Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG			
Aspect	RAG	GraphRAG	Agentic RAG
 Approche principale	Récupération de documents pertinents pour augmenter la réponse du LLM. 	Exploite la structure des données et les relations sémantiques (graphe). 	Utilise un agent autonome pour planifier, rechercher, agir et synthétiser. 
 Représentation des données	Documents non structurés (textes, PDFs, etc.) 	Grappe de connaissances (entités, relations, événements). 	Documents + outils + mémoire + instructions pour l'agent. 
 Récupération d'information	Recherche sémantique par similarité vectorielle (embeddings). 	Navigation dans le graphe (requêtes sur entités et relations). 	L'agent décide quoi chercher, où chercher et comment combiner les infos. 
 Raisonnement	Limité au contexte récupéré + capacités du LLM. 	Raisonnement basé sur les relations et la structure du graphe. 	Raisonnement multi-étapes, planification et adaptation autonome. 
 Orchestration	Pipeline fixe : Retrieval → Augmentation → Generation 	Pipeline avec requêtes sur le graphe + génération. 	Boucle agentique : Planifier → Agir → Observer → Réajuster → Répondre. 
 Cas d'usage typiques	FAQ, recherche documentaire, chatbots de base. 	Analyse relationnelle, détection d'insights, contexte complexe. 	Tâches complexes, multi-sources, prise de décision, workflows autonomes. 

La figure 1.2 distingue trois niveaux fonctionnels. La RAG récupère surtout des documents proches d’une requête et les transmet au LLM ; elle reste donc efficace pour une recherche documentaire, mais son raisonnement dépend du contexte retrouvé. Le GraphRAG ajoute une représentation relationnelle : les entités et leurs liens permettent de naviguer entre compétences, métiers, offres, niveaux et formations. L’Agentic RAG va plus loin : il ne se limite pas à récupérer une information, il planifie les étapes, choisit les outils, observe les résultats, ajuste le contexte et produit une réponse. Cette distinction justifie le choix du mémoire : combiner retrieval vectoriel, graphe Neo4j et orchestration LangGraph pour traiter des requêtes emploi-compétences plus complexes qu’une simple recherche de documents.

1.5.2 Agentic RAG, critic et traçabilité

Les travaux ReAct montrent l’intérêt d’alterner raisonnement et action outillée dans les modèles de langage [35]. Self-RAG va plus loin en introduisant une logique d’auto-réflexion pour décider quand récupérer de l’information et comment critiquer la réponse [36]. Pour ce mémoire, ces travaux ne doivent pas être interprétés comme une autorisation à laisser un agent

décider librement. Ils justifient plutôt une orchestration contrôlée : classifier l'intention, choisir les sources, récupérer les preuves, construire le contexte, générer, puis vérifier si la réponse est suffisamment fondée.

Le manque de la littérature est ici opérationnel. Beaucoup de travaux décrivent des architectures générales, mais la fiabilité dépend de détails concrets : qualité des chunks, schéma du graphe, robustesse des requêtes, traçabilité des sources, gestion des cas sans résultat et critères de révision. C'est pourquoi l'architecture retenue ajoute un critic et des traces d'exécution. Le LLM produit la réponse finale, mais les données récupérées et les outils appelés doivent rester observables.

1.5.3 Évaluer la performance et la factualité

L'évaluation d'un système emploi-compétences ne peut pas se limiter à la fluidité de la réponse générée. Elle doit d'abord mesurer la qualité de la récupération d'information : le système retrouve-t-il les offres, compétences ou documents réellement pertinents ? La littérature en recherche d'information utilise pour cela des métriques de classement comme Precision@k, Recall@k, MRR et NDCG [27, 28]. Ces métriques supposent l'existence d'un ensemble de résultats pertinents, noté $Rel(q)$, pour une requête q , et d'une liste classée de résultats retournés par le système.

La **Precision@k** mesure la proportion de résultats pertinents parmi les k premiers résultats retournés. Si $R_k(q)$ désigne l'ensemble des k premiers éléments proposés pour la requête q , alors :

$$Precision@k(q) = \frac{|R_k(q) \cap Rel(q)|}{k}. \quad (1.1)$$

Cette métrique répond à la question suivante : parmi les premières recommandations affichées, combien sont réellement utiles ? Dans un système emploi-compétences, une Precision@5 élevée signifie que les cinq premières offres proposées contiennent peu de résultats non pertinents. Sa limite est qu'elle ne mesure pas si le système a retrouvé tous les bons résultats disponibles.

Le **Recall@k** mesure au contraire la proportion des éléments pertinents retrouvés dans les k premiers résultats :

$$Recall@k(q) = \frac{|R_k(q) \cap Rel(q)|}{|Rel(q)|}. \quad (1.2)$$

Cette métrique répond à la question : parmi toutes les offres ou compétences pertinentes,

quelle part le système a-t-il réussi à faire remonter ? Elle est importante pour ne pas éliminer trop tôt des offres utiles. En recommandation emploi, un faible Recall@k peut signifier que le moteur rate des opportunités pertinentes pour le candidat.

Le **MRR** (*Mean Reciprocal Rank*) évalue la position du premier résultat pertinent. Pour une requête q , si $rank_q$ est le rang du premier résultat pertinent, alors le score associé est :

$$RR(q) = \frac{1}{rank_q}. \quad (1.3)$$

Sur un ensemble de N requêtes, le MRR est :

$$MRR = \frac{1}{N} \sum_{q=1}^N \frac{1}{rank_q}. \quad (1.4)$$

Cette métrique est utile lorsque l'on veut savoir si le système place rapidement au moins une bonne réponse. Dans une interface de recommandation, cela correspond à une logique pratique : l'utilisateur regarde souvent les premiers résultats avant de parcourir toute la liste.

Le **NDCG@k** (*Normalized Discounted Cumulative Gain*) tient compte non seulement de la présence des résultats pertinents, mais aussi de leur ordre et éventuellement de leur degré de pertinence. Si rel_i désigne le niveau de pertinence du résultat placé au rang i , alors :

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}. \quad (1.5)$$

Le score est ensuite normalisé par le meilleur classement possible, noté $IDCG@k$:

$$NDCG@k = \frac{DCG@k}{IDCG@k}. \quad (1.6)$$

Cette métrique est particulièrement adaptée lorsque tous les résultats pertinents ne se valent pas. Par exemple, une offre parfaitement alignée avec les compétences du candidat devrait être mieux classée qu'une offre seulement partiellement compatible.

Dans une architecture RAG, l'évaluation ne s'arrête pas au classement des résultats. Elle doit aussi mesurer la qualité de la réponse générée à partir du contexte récupéré. RAGAS distingue notamment la fidélité au contexte, la pertinence de la réponse, la qualité de la récupération et l'usage des sources [37]. La **fidélité** vérifie si les affirmations de la réponse sont supportées par les éléments récupérés. La **pertinence de réponse** vérifie si la réponse traite effectivement la question posée. La **couverture des sources** vérifie si le contexte contient suffisamment d'informations pour répondre. Enfin, la **traçabilité** vérifie si les preuves ou citations

sont identifiables.

Ces critères sont moins simples à formaliser que Precision@k ou Recall@k, car ils portent sur du langage naturel. Des approches comme G-Eval utilisent des LLM comme juges pour approximer l'évaluation humaine [38]. Cette pratique doit rester prudente : un juge automatique peut aider à comparer des variantes de système, mais il ne remplace pas une validation métier. Dans ce mémoire, l'évaluation doit donc combiner des métriques de retrieval pour pgvector et Neo4j, des critères de factualité pour la réponse générée, et une interprétation professionnelle de l'utilité des recommandations.

1.6 Positionnement de l'étude

Les travaux présentés dans cette revue montrent que le matching emploi-compétences ne peut pas être traité comme une simple recherche de mots-clés. Il exige à la fois une représentation sémantique des textes, une structuration des relations métier-compétence-formation et un mécanisme de génération capable d'expliquer les résultats sans s'éloigner des données disponibles.

Cette étude se positionne à l'intersection de trois familles d'approches. La première est la recherche sémantique par embeddings, utilisée pour retrouver rapidement les offres, compétences ou documents proches d'une requête. La deuxième est le graphe de connaissances, mobilisé pour représenter les relations entre candidats, offres, compétences, métiers, secteurs et niveaux de formation. La troisième est l'orchestration agentique, qui organise le retrieval, l'enrichissement par le graphe, la génération et le contrôle de la réponse.

La contribution du mémoire n'est donc pas de proposer un nouveau modèle de langage ou un nouvel algorithme de recommandation générique. Elle consiste à concevoir et évaluer une architecture hybride adaptée au contexte emploi-compétences : pgvector sert à la présélection sémantique, Neo4j apporte le signal relationnel et LangGraph orchestre les étapes de décision. Ce positionnement répond à une limite de la littérature : les approches générales de RAG ou de recommandation expliquent rarement comment articuler, dans un même système opérationnel, données locales, référentiels métiers, graphe, scoring hybride et contrôle de la génération.

Le système proposé reste un outil d'aide à la décision. Il ne remplace ni le recruteur, ni le conseiller emploi. Son rôle est de rendre les correspondances plus lisibles : quelles offres sont proches du profil, quelles compétences expliquent le rapprochement, quels écarts subsistent et quelles pistes de montée en compétence peuvent être envisagées.

Sources de données et approche méthodologique

Ce chapitre précise les données mobilisées et la logique de construction du système. Il ne présente pas encore les résultats : il explique comment les sources sont préparées, comment elles sont représentées, puis comment elles sont interrogées par un workflow hybride.

2.1 Cadre méthodologique et sources de données

Cette première section fixe le cadre de travail. Le système est conçu comme un artefact d'aide à la décision, pas comme un modèle isolé.

2.1.1 Démarche d'ingénierie appliquée retenue

La démarche retenue est une démarche d'ingénierie appliquée. Elle vise à construire un système de recommandation emploi-compétences combinant données structurées, embeddings, graphe de connaissances, règles métier et RAG. Ce choix répond directement au problème étudié : les offres, profils, compétences et référentiels sont hétérogènes et rarement comparables tels quels.

Quatre principes guident la méthode. La traçabilité impose de rattacher chaque recommandation à des données ou relations observables. L'hybridation combine pgvector, Neo4j et règles métier au lieu de dépendre d'une seule source. L'explicabilité oblige le système à signaler les compétences communes et manquantes. Le contrôle génératif limite le rôle du LLM à une réponse fondée sur le contexte récupéré, conformément à la logique du RAG [4, 37].

LangGraph est retenu comme cadre d'orchestration du workflow agentique [39]. Son rôle méthodologique est de router les requêtes, conserver les traces des outils appelés et permettre une révision lorsque le contexte récupéré est insuffisant.

Le workflow complet est reporté en annexe à la figure A.1. Il suit une logique simple : la requête est analysée, pgvector retrouve les éléments proches, Neo4j vérifie les relations métier-compétence-formation, puis le LLM formule une réponse contrôlée par le critic. Le tableau A.4, également placé en annexe, résume la correspondance entre objectifs spécifiques et choix méthodologiques.

2.1.2 Sources de données et leur rôle dans le système

Les données se répartissent en deux groupes. Les données opérationnelles décrivent les offres et les candidats. Les référentiels ESCO, MEPC et NCF apportent les nomenclatures nécessaires pour interpréter les métiers, compétences et niveaux de formation.

Les offres décrivent les postes disponibles : titre, employeur, secteur, localisation, expérience, niveau attendu, compétences et description. Les profils candidats décrivent le diplôme, le niveau, l'expérience, les compétences, la mobilité et l'objectif professionnel. Ces informations sont utiles mais bruitées ; elles doivent donc être normalisées avant l'encodage et la construction du graphe.

Table 2.1 – Sources de données mobilisées et utilisation méthodologique

Source	Nature	Utilisation dans la méthode
Offres d'emploi	Données opérationnelles collectées par KmerAI	Normalisation, extraction de compétences, génération de <code>text_to_embed</code> , indexation vectorielle, création des noeuds d'offres et calcul des scores de matching.
Profils candidats	Base locale de demandeurs	Normalisation du niveau, du diplôme, de la mobilité et du métier visé ; création des embeddings candidats ; analyse de compatibilité avec les offres.
ESCO	Référentiel métiers-compétences	Source de métiers, compétences, groupes ISCO et relations métier-compétence, utilisée pour structurer le graphe et enrichir les correspondances.
MEPC	Nomenclature camerounaise des métiers	Ancrage national des métiers, structuration hiérarchique et alignement partiel avec les groupes internationaux.
NCF	Nomenclature camerounaise des formations	Codification des niveaux et domaines de formation, utilisée pour la compatibilité candidat-offre et l'analyse de montée en compétences.
PDF réglementaires	Documents officiels fragmentés	Constitution de <code>DocChunk</code> pour la recherche référentielle et la citation des sources.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

2.1.3 Préparation et normalisation des données

La normalisation prépare les données pour deux usages : l’encodage vectoriel et la création des relations Neo4j. Elle réduit les doublons, harmonise les champs textuels, sépare les valeurs multiples et transforme les compétences en listes exploitables.

Pour les candidats, le traitement porte surtout sur le diplôme, le niveau d’étude, l’expérience, les compétences, le secteur visé et la mobilité. Lorsque c’est possible, le niveau est rapproché de la NCF afin de ne pas confondre proximité textuelle et compatibilité de qualification.

Chaque offre et chaque candidat reçoit un champ `text_to_embed`. Il regroupe les informations utiles à la recherche sémantique : titre, compétences, secteur, niveau, description et objectif.

2.1.4 Alignement des référentiels métiers et formations

L’alignement rend comparables les annonces, les profils et les référentiels. La MEPC structure les métiers selon ses niveaux hiérarchiques ; la NCF structure les niveaux et domaines de formation ; ESCO apporte un vocabulaire international métier-compétence.

L’alignement repose sur les codes disponibles et les proximités de libellés. Il sert à enrichir la couverture et l’interprétabilité du système, sans prétendre établir une équivalence parfaite entre toutes les nomenclatures.

2.2 Représentation sémantique et graphe de connaissances

Cette section décrit les représentations utilisées par le système : embeddings pour la proximité textuelle, pgvector pour le retrieval, Neo4j pour les relations métier et score hybride pour le classement.

2.2.1 Choix du modèle d’embeddings et fine-tuning

Le modèle SentenceTransformer représente les offres, candidats, métiers, compétences et documents dans un espace vectoriel. Ce choix est adapté à la recherche sémantique, car les embeddings peuvent être comparés par similarité cosinus [23].

Le fine-tuning n’est pas génératif. Il adapte l’espace vectoriel au vocabulaire du domaine par apprentissage contrastif : rapprocher les paires pertinentes et éloigner les paires moins pertinentes. Les paires d’apprentissage sont construites à partir des relations et champs normalisés, puis le modèle adapté est évalué séparément du modèle de base.

2.2.2 Indexation vectorielle dans PostgreSQL et pgvector

Les embeddings sont stockés dans PostgreSQL avec pgvector, afin de combiner vecteurs et métadonnées dans la même base [2]. L'indexation de type HNSW sert à accélérer la recherche top-K [29].

pgvector joue deux rôles : retrouver les objets proches d'une requête et constituer un vivier initial avant le reranking par le graphe. Chaque résultat conserve son identifiant, son type, sa source, son score et le texte associé pour préserver la traçabilité.

2.2.3 Construction du graphe de connaissances sous Neo4j

Neo4j représente la couche relationnelle du système. Les noeuds correspondent aux offres, candidats, compétences, métiers, secteurs, localisations et référentiels. Les relations décrivent les liens utiles au matching, par exemple `REQUIERT` entre offre et compétence ou `POSSEDE` entre candidat et compétence.

Le graphe sert à vérifier les correspondances proposées par pgvector, expliquer les compétences communes ou manquantes et répondre à des questions relationnelles que la recherche vectorielle traite mal.

2.2.4 Score hybride et logique de skill gap

Le score de recommandation combine des signaux complémentaires : similarité sémantique, couverture des compétences, compatibilité de niveau et alignement sectoriel.

La forme générale du score est la suivante :

$$S_{hyb}(c, o) = \alpha S_{sem}(c, o) + \beta S_{graph}(c, o) + \gamma C_{ncf}(c, o) + \delta A_{sect}(c, o),$$

où c désigne un candidat, o une offre, S_{sem} la similarité sémantique, S_{graph} la couverture des compétences, C_{ncf} la compatibilité de niveau et A_{sect} l'alignement sectoriel. Le LLM n'intervient donc pas dans le calcul brut du classement ; il sert à synthétiser et expliquer.

2.2.5 Construction du proxy de pertinence pour l'évaluation

En l'absence d'annotations indépendantes de recruteurs, l'évaluation utilise un label faible nommé `proxy_relevance`. Ce proxy ne fait pas partie du score hybride utilisé par le moteur ; il sert uniquement de référence expérimentale pour comparer les classements.

Pour un candidat c et une offre o , le proxy combine trois informations issues des métadonnées normalisées :

$$R_{\text{proxy}}(c, o) = 0,45L(c, o) + 0,30S(c, o) + 0,25N(c, o),$$

où $L(c, o)$ mesure le recouvrement lexical, $S(c, o)$ l'alignement sectoriel et $N(c, o)$ la compatibilité de niveau NCF. Le recouvrement lexical est défini par :

$$L(c, o) = \frac{|T_c \cap T_o|}{\max(\min(|T_c|, |T_o|), 1)}.$$

La compatibilité NCF vaut 1 si le niveau du candidat couvre le niveau demandé, 0,70 pour un écart d'un niveau, 0,40 pour deux niveaux et 0,10 au-delà. Si l'un des niveaux est absent, une valeur neutre de 0,45 évite de transformer une donnée manquante en incompatibilité certaine.

Le score continu R_{proxy} est utilisé comme gain gradué dans le calcul du NDCG. Pour Precision@10, Recall@10 et MRR@10, il est transformé en label binaire séparément pour chaque candidat. Une offre est déclarée pertinente lorsque :

$$R_{\text{proxy}}(c, o) \geq \max(0,35, Q_{0,70,c}),$$

où $Q_{0,70,c}$ est le 70^e percentile des scores du vivier du candidat. Ce protocole est reproductible, mais il reste un proxy : il mesure une cohérence interne avec les métadonnées disponibles, pas une validation métier définitive.

Les pondérations sont calibrées avec Optuna, par optimisation bayésienne et *Tree-structured Parzen Estimator* [40]. Dans l'expérience retenue, la calibration porte sur deux signaux : score sémantique pgvector et score Neo4j.

$$S_{\text{cal}}(c, o) = w_{\text{sem}}S_{\text{sem}}(c, o) + w_{\text{Neo4j}}S_{\text{Neo4j}}(c, o), \quad w_{\text{sem}} + w_{\text{Neo4j}} = 1.$$

À chaque essai, Optuna propose w_{sem} , déduit $w_{\text{Neo4j}} = 1 - w_{\text{sem}}$, recalcule les scores, re-range les offres et évalue le classement. Trois objectifs sont comparés : NDCG@10, moyenne NDCG@10–Recall@10–Precision@10, et moyenne MRR@10–Precision@10. Les poids obtenus sont donc optimaux dans ce protocole expérimental, non des paramètres définitifs du marché du travail.

L'analyse de *skill gap* complète le score en distinguant compétences acquises, requises et manquantes. Elle transforme le classement en verdict opérationnel : compatibilité immédiate, compatibilité après montée en compétence, vivier ou profil hors cible.

2.3 Orchestration agentique et génération contrôlée

Cette section présente le rôle du workflow agentique dans la méthode. Les détails d'implémentation sont développés au chapitre 3.

2.3.1 Architecture Agentic RAG et orchestration LangGraph

LangGraph représente l'agent comme un graphe d'états plutôt que comme une succession libre de prompts [39]. La requête est analysée, classée par cas d'usage, puis dirigée vers les outils adaptés : pgvector, Neo4j, recherche documentaire, score hybride ou diagnostic.

Ce choix relève de l'Agentic RAG : l'agent orchestre des outils spécialisés autour d'un état partagé [35, 36]. Dans ce mémoire, l'agent reste borné : il route, trace et contrôle ; il ne remplace pas les bases ni les règles métier.

2.3.2 Text2Cypher et rôle du LLM

Deux usages du LLM sont distingués. Text2Cypher transforme certaines questions relationnelles en requêtes Cypher, sous contrainte du schéma Neo4j. La génération finale formule ensuite une réponse française à partir du contexte récupéré.

Cette séparation est importante : le LLM ne décide pas seul de la pertinence. Il interroge ou rédige à partir des données disponibles.

2.3.3 Modes d'accès et traçabilité des décisions

Trois modes d'accès sont prévus : CLI pour les tests, Streamlit pour la démonstration conversationnelle et FastAPI pour l'intégration applicative. Ces interfaces exposent le même moteur.

La traçabilité repose sur les traces du workflow, les identifiants des entités, les sources documentaires, les scores et les chemins du graphe. Une recommandation sans preuve n'est pas défendable dans un contexte d'orientation ou de recrutement.

2.3.4 Protocole d'évaluation prévu du système

L'évaluation prévue sépare quatre niveaux : retrieval vectoriel, validation fonctionnelle du graphe, comparaison des classements et contrôle de la génération. Les métriques de ranking retenues incluent Precision@K, Recall@K, MRR@K, MAP@K et NDCG@K [27, 28].

Cette séparation évite une confusion classique dans les systèmes RAG : une réponse bien formulée ne prouve pas que les bonnes informations ont été retrouvées ; inversement, un bon retrieval ne garantit pas une bonne synthèse.

Deuxième partie

Présentation des résultats

Construction opérationnelle du système hybride emploi-compétences

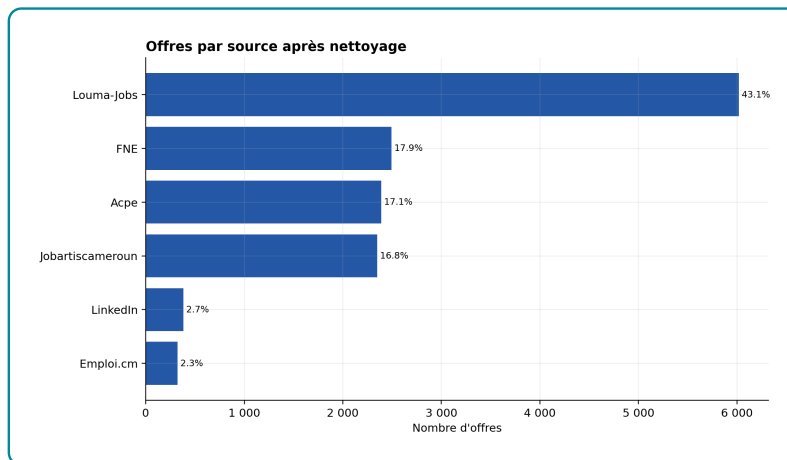
Ce chapitre suit le workflow réellement implémenté : préparation des données, adaptation de l'encodeur, indexation dans pgvector, construction du graphe Neo4j et calcul du score hybride. Seuls les résultats nécessaires pour justifier ces choix techniques sont présentés ; leur évaluation comparative est réservée au chapitre 4.

3.1 Données exploitables et contraintes de qualité du système

3.1.1 Volumes disponibles après nettoyage

L'ETL produit les offres, profils et référentiels normalisés qui alimentent l'encodeur, pgvector et Neo4j. Le corpus compte 13 957 offres pour 41 298 candidats. Ce rapport déséquilibré augmente le risque de recommander selon la fréquence des secteurs ou des compétences ; il justifie l'ajout d'un contrôle relationnel après le rappel sémantique. Les volumes complets figurent au tableau A.5 en annexe.

Figure 3.1 – Répartition des offres par source après nettoyage



Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Les offres proviennent de plateformes scrapées et de sources institutionnelles. Louma-Jobs représente 43,14 % du corpus : les résultats peuvent refléter les caractéristiques de cette source et ne décrivent pas exhaustivement le marché camerounais de l'emploi.

3.1.2 Qualité des variables utiles au matching emploi-compétences

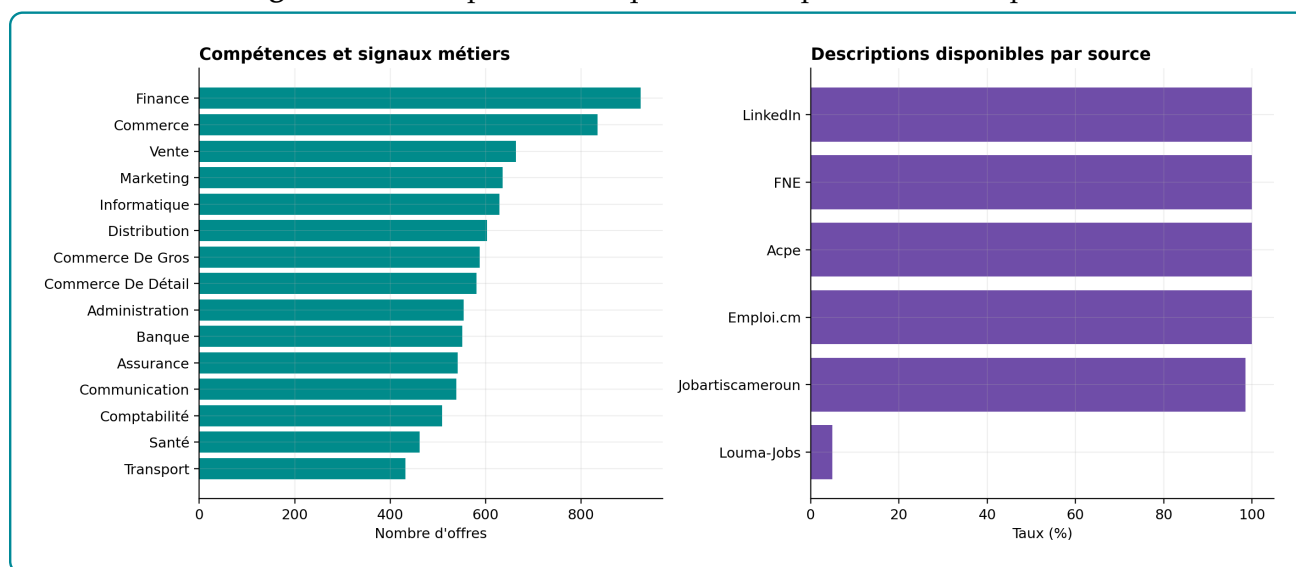
Le matching exploite la description, les compétences, la localisation, le secteur et le niveau. La source reste une variable de traçabilité et permet d'interpréter les écarts de qualité entre plateformes.

3.1.2.1 Localisation exploitable pour filtrer sans exclure

La localisation est utilisée comme filtre souple : 79,88 % des offres sont rattachées au Cameroun et 20,12 % à l'international. Une ville absente n'est pas imputée ; le pays disponible est conservé. Le diagnostic détaillé figure en annexe (figure A.3).

3.1.2.2 Compétences et descriptions : signal utile mais hétérogène

Figure 3.2 – Compétences fréquentes et disponibilité descriptive



Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Les libellés fréquents mélangent compétences observables et domaines d'activité ; ils sont utilisés comme indices de rapprochement, non comme preuve de maîtrise. Louma-Jobs ne fournit une description que pour 4,90 % de ses offres. Pour éviter qu'une description absente produise un vecteur pauvre, le texte encodé combine titre, compétences, secteur, localisation et autres métadonnées disponibles.

3.1.3 Nettoyage réalisé et limites conservées

Le nettoyage harmonise les formats sans inventer les valeurs absentes. Les localisations suivent la priorité ville-pays et les doublons potentiels sont audités sans suppression automatique, car deux annonces proches peuvent correspondre à des publications distinctes. Les

règles détaillées figurent au tableau A.6 en annexe.

Le pipeline peut traiter de nouvelles offres et de nouveaux profils dès lors que les colonnes minimales sont respectées. Une nouvelle exécution applique les mêmes contrôles, régénère les fichiers normalisés, puis permet de réencoder et synchroniser les nouvelles entités dans pgvector et Neo4j. Il s'agit d'une actualisation par lots contrôlée, non d'une ingestion en temps réel ; un changement du format d'une source scrapée exige d'adapter son mapping.

3.2 Adaptation du modèle d'embeddings au domaine emploi-compétences

Après l'ETL, les textes nettoyés servent à adapter l'encodeur avant leur indexation dans pgvector. L'objectif est d'améliorer le classement sémantique, non de générer une réponse.

3.2.1 Encodeur retenu et paires contrastives

Le modèle all-MiniLM-L6-v2 a été retenu pour ses vecteurs de 384 dimensions, moins coûteux à encoder, stocker et rechercher que des représentations plus larges. Son fine-tuning adapte cet espace généraliste aux intitulés, secteurs, niveaux et compétences du corpus local sans modifier l'architecture de production.

Table 3.1 – Protocole de construction des paires de fine-tuning

Élément	Implémentation observée
Modèle de départ	sentence-transformers/all-MiniLM-L6-v2, embeddings denses de 384 dimensions.
Rôle de <code>sentence1</code>	Représentation structurée de l'offre : titre, secteur, contrat, niveau d'études, expérience, ville et compétences.
Rôle de <code>sentence2</code>	Détails nettoyés de l'annonce uniquement, c'est-à-dire le texte descriptif que le modèle doit rapprocher de la représentation structurée.
Filtrage informatif	Conservation des paires avec compétences disponibles, <code>sentence1</code> d'au moins 40 caractères et <code>sentence2</code> d'au moins 120 caractères; le champ <code>ft_eligible</code> est appliqué lorsqu'il existe.
Nombre total de paires	6 476 paires offre-description.
Split	4 542 paires en train, 981 en validation et 953 en test, avec stratification par secteur principal.

Source : Nos travaux, traitements Python et SentenceTransformer.

`sentence1` contient la représentation structurée de l'offre et `sentence2` sa description. Les seuils de 40 et 120 caractères excluent les paires trop pauvres pour fournir un signal d'apprentissage exploitable.

3.2.2 Fine-tuning : contraintes de calcul et preuve de gain

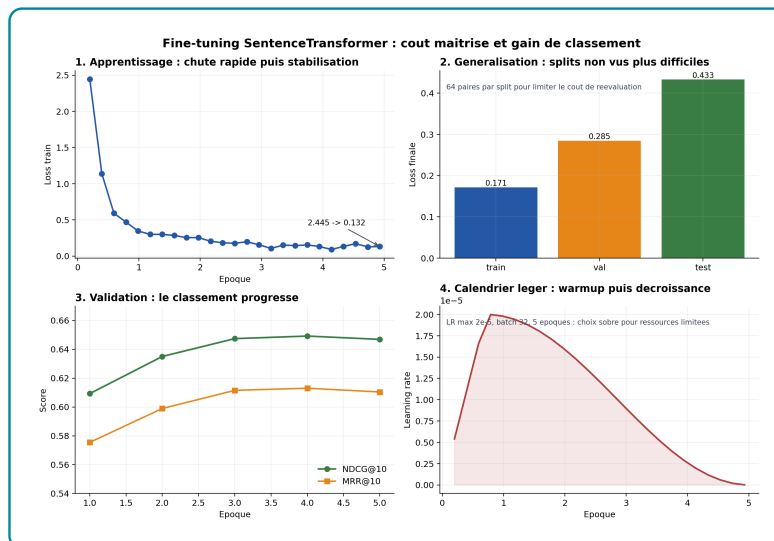
3.2.2.1 Paramétrage contraint et dynamique d'apprentissage

Table 3.2 – Hyperparamètres du fine-tuning SentenceTransformer

Hyperparamètre	Valeur
Nombre d'époques	5
Batch size	32
Négatifs implicites par exemple	31
Learning rate maximal	2×10^{-5}
Scheduler	Cosine avec warmup
Warmup	100 étapes configurées, limitées par les étapes disponibles de l'époque
Perte	<i>MultipleNegativesRankingLoss</i> avec similarité cosinus
Temps d'entraînement observé	15 160 s
Checkpoint retenu	Meilleur checkpoint sur validation autour de l'époque 4

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Ces choix répondent aux ressources disponibles. Les vecteurs de 384 dimensions réduisent le stockage et le coût de recherche ; le batch de 32 limite la mémoire tout en fournissant 31 négatifs implicites par exemple ; cinq époques contiennent le temps d'entraînement. Le *learning rate* de 2×10^{-5} évite une modification trop brutale des représentations initiales.

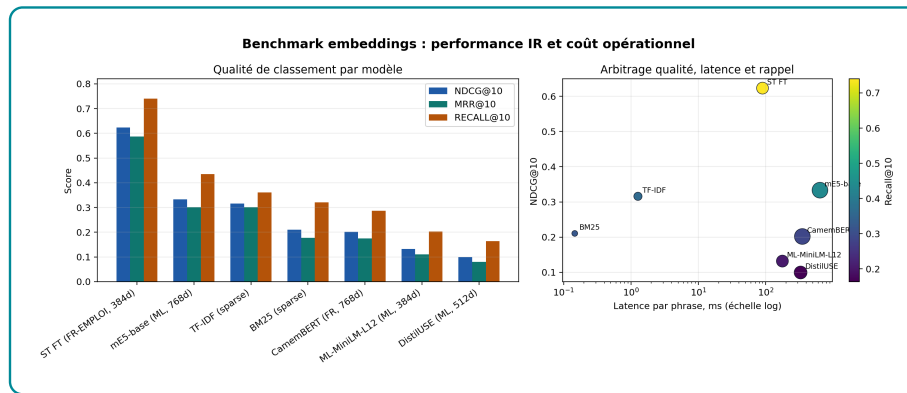
Figure 3.3 – Diagnostic du fine-tuning : perte, généralisation, métriques de validation et calendrier du learning rate

Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.

La perte d'entraînement diminue de 2,445 à 0,132, tandis que la perte de test reste plus élevée (0,433) que celle d'entraînement (0,171). Cet écart signale une généralisation moins bonne sur les paires non vues, sans divergence du modèle. Le panneau des métriques montre un NDCG@10 maximal à l'époque 4, puis un léger recul ; le panneau du learning rate confirme une montée puis une décroissance contrôlée du pas d'apprentissage. Prolonger l'entraînement n'était donc pas justifié.

3.2.2.2 Preuve de gain sur le ranking sémantique

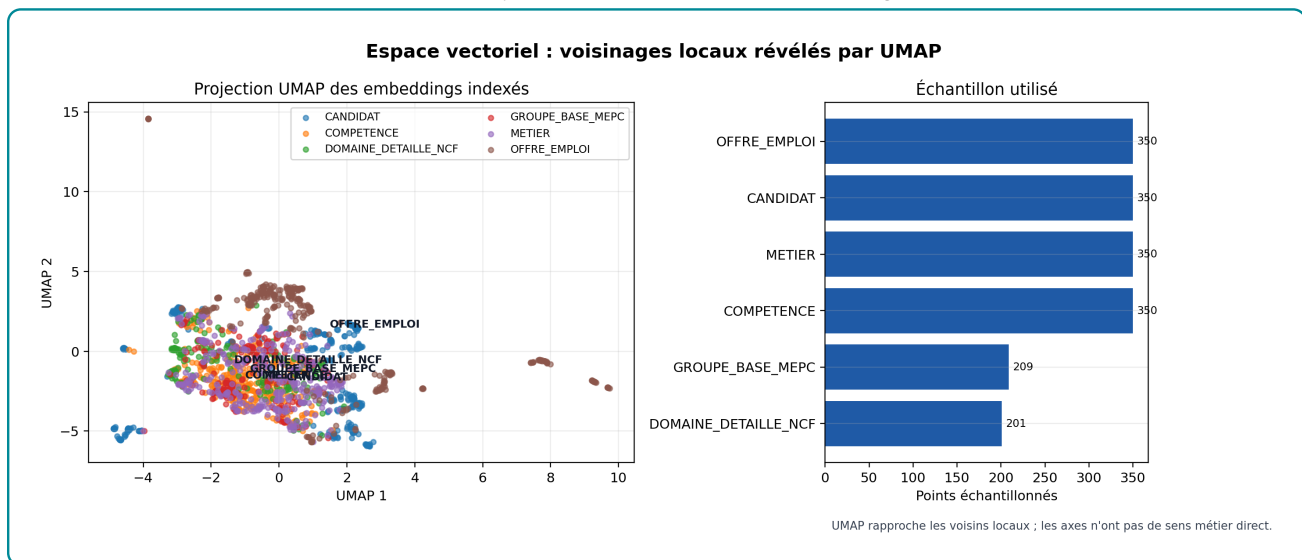
Sur 953 paires de test, le fine-tuning multiplie le NDCG@10 par 3,7 et le MRR@10 par 4,2 par rapport au modèle de départ. Le détail figure au tableau A.7 en annexe ; l'évaluation complète est présentée au chapitre 4.

Figure 3.4 – Comparaison des modèles d'embeddings : qualité de classement, rappel et latence

Source : Nos travaux, traitements Python et SentenceTransformer

Le modèle fine-tuné obtient un NDCG@10 de 0,6232, un MRR@10 de 0,5874 et un Recall@10 de 0,7398. Les barres de qualité de classement et de rappel justifient son choix pour pgvector ; la latence rappelle toutefois que le meilleur modèle statistique doit rester compatible avec une recherche interactive. TF-IDF reste un complément rapide pour les correspondances lexicales exactes.

3.2.3 Structure de l'espace vectoriel après fine-tuning

Figure 3.5 – Projection UMAP des embeddings indexés

Source : Nos travaux, traitements Python et SentenceTransformer

UMAP projette 1 810 embeddings indexés. Les offres sont plus excentrées parce que leurs textes combinent titre, contexte et contraintes. Les candidats, compétences, métiers et référentiels se recouvrent davantage : cette proximité permet de retrouver des objets de types

différents portant sur le même domaine.

UMAP ne mesure toutefois ni la pertinence ni la distance exacte dans l'espace original de 384 dimensions. Il fournit un contrôle visuel ; les métriques de ranking mesurent la qualité du retrieval et Neo4j vérifie ensuite les relations métier.

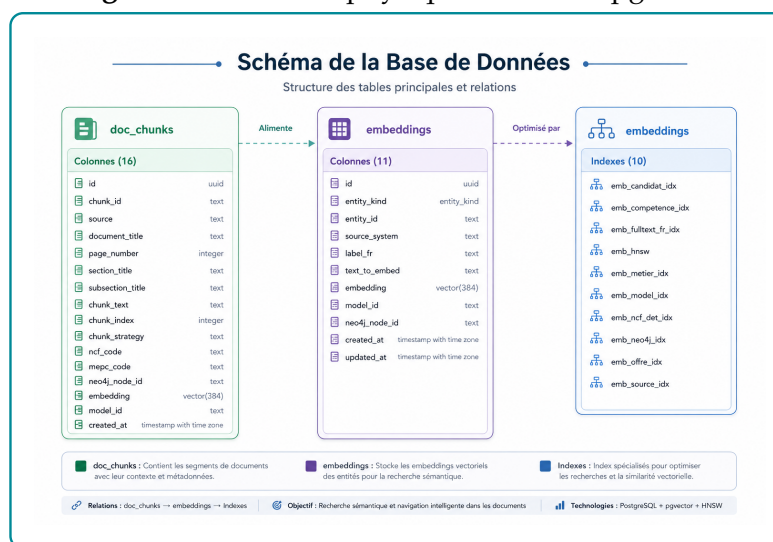
3.3 Mémoire vectorielle pgvector pour le retrieval

Après l'encodage, pgvector récupère un top-K d'objets sémantiquement proches. Ce vivier est ensuite contrôlé et rerangé par Neo4j.

3.3.1 Schéma physique, tables et volumes indexés

La table `embeddings` stocke les entités structurées ; `doc_chunks` contient les fragments des référentiels. Le schéma montre aussi le rôle des index : accélérer la similarité vectorielle, la recherche textuelle et les filtres par type d'entité. L'identifiant commun assure ensuite le passage vers Neo4j.

Figure 3.6 – Schéma physique de la base pgvector



Source : Nos travaux, diagnostics PostgreSQL-pgvector

L'instance contient 72 643 embeddings d'entités et 142 embeddings documentaires, tous en 384 dimensions. La couverture Neo4j de 100 % signifie que chaque résultat vectoriel peut être remplacé dans le graphe pour récupérer ses relations métier. Elle mesure la présence d'un lien technique, pas la qualité sémantique de ce lien.

Table 3.3 – Distribution des embeddings par type d'entité

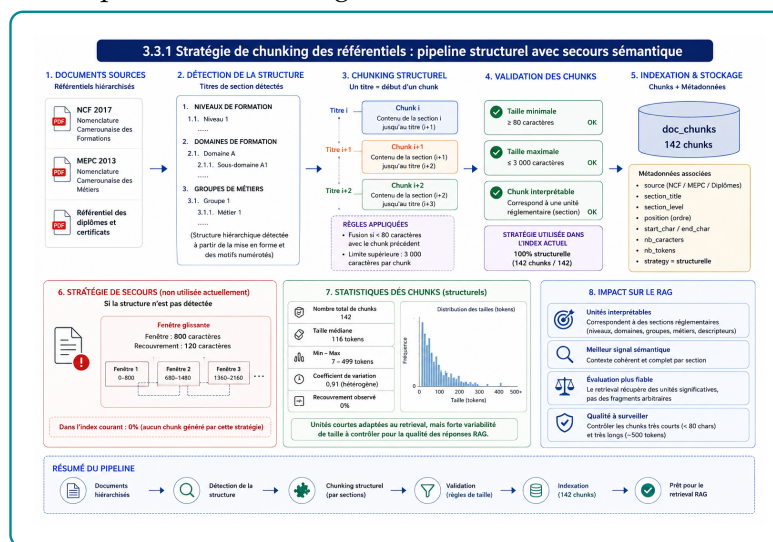
Type d'entité	Vecteurs	Part	Couverture Neo4j
Candidats	41 298	56,85 %	100 %
Offres d'emploi	13 957	19,22 %	100 %
Compétences	13 939	19,19 %	100 %
Métiers	3 039	4,18 %	100 %
Groupes de base MEPC	209	0,29 %	100 %
Domaines détaillés NCF	201	0,28 %	100 %
Total entités structurées	72 643	100 %	100 %
Chunks documentaires	142	–	100 %

Source : Nos travaux, diagnostics PostgreSQL-pgvector.

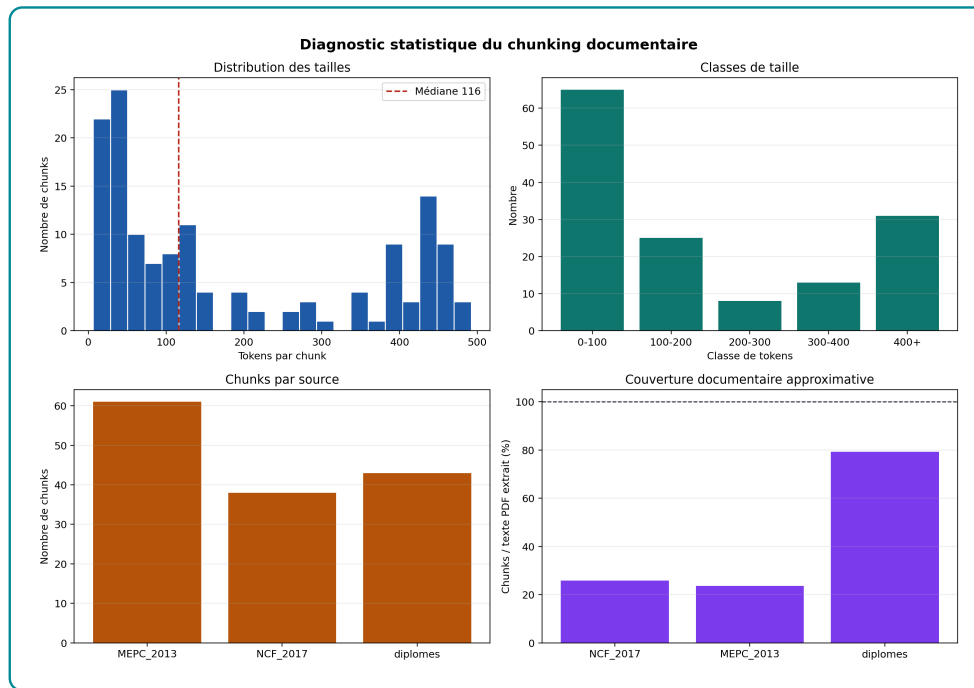
3.3.2 Référentiels découpés en chunks interrogeables

Les référentiels produisent 142 chunks : 43 pour les diplômes, 61 pour le MEPC 2013 et 38 pour la NCF 2017. La figure du pipeline montre cinq étapes : identification des documents, détection des titres, découpage structurel, validation des tailles et stockage dans pgvector. Les fragments de moins de 80 caractères sont fusionnés et la taille maximale est fixée à 3 000 caractères. La stratégie de secours par fenêtres glissantes n'a pas été utilisée dans l'index courant.

Figure 3.7 – Pipeline de chunking structurel des référentiels documentaires



Source : Nos travaux, schématisation du pipeline de chunking documentaire et d'indexation pgvector.

Figure 3.8 – Distribution des tailles, sources et couverture du chunking documentaire

Source : Nos travaux, diagnostics Python du chunking documentaire.

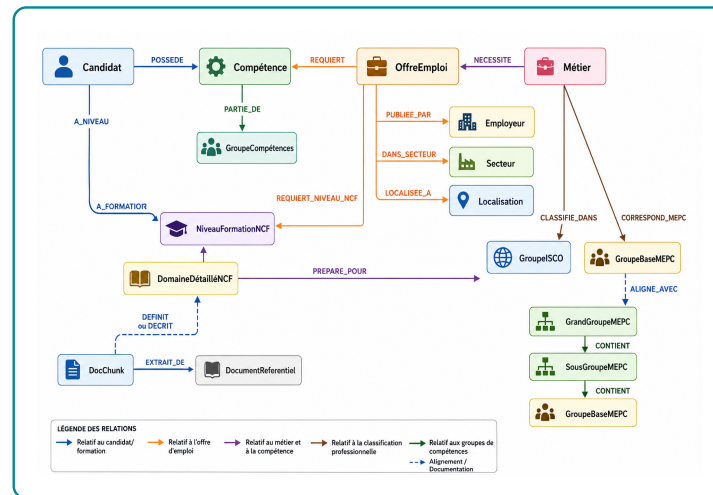
La taille médiane est de 116 tokens, mais certains fragments dépassent 400 tokens. Les chunks courts risquent de manquer de contexte ; les plus longs peuvent mélanger plusieurs idées. La couverture moyenne de 42,9 % correspond aux tokens conservés, non à la proportion exacte d'information utile. Enfin, la cohérence moyenne plus faible des diplômes (0,268 contre 0,336 pour la NCF) signale que leur découpage doit être contrôlé en priorité. Les mesures complètes figurent en annexe.

3.4 Neo4j : mémoire relationnelle du matching

Dans le workflow méthodologique, Neo4j intervient après le rappel pgvector et avant le score hybride. pgvector propose des objets proches dans l'espace sémantique ; Neo4j vérifie ensuite si cette proximité repose sur des relations métier exploitables : compétences possédées, compétences requises, métiers, niveaux NCF, secteurs et localisations. Cette section présente donc la structure du graphe et les précautions nécessaires avant son usage dans le reranking.

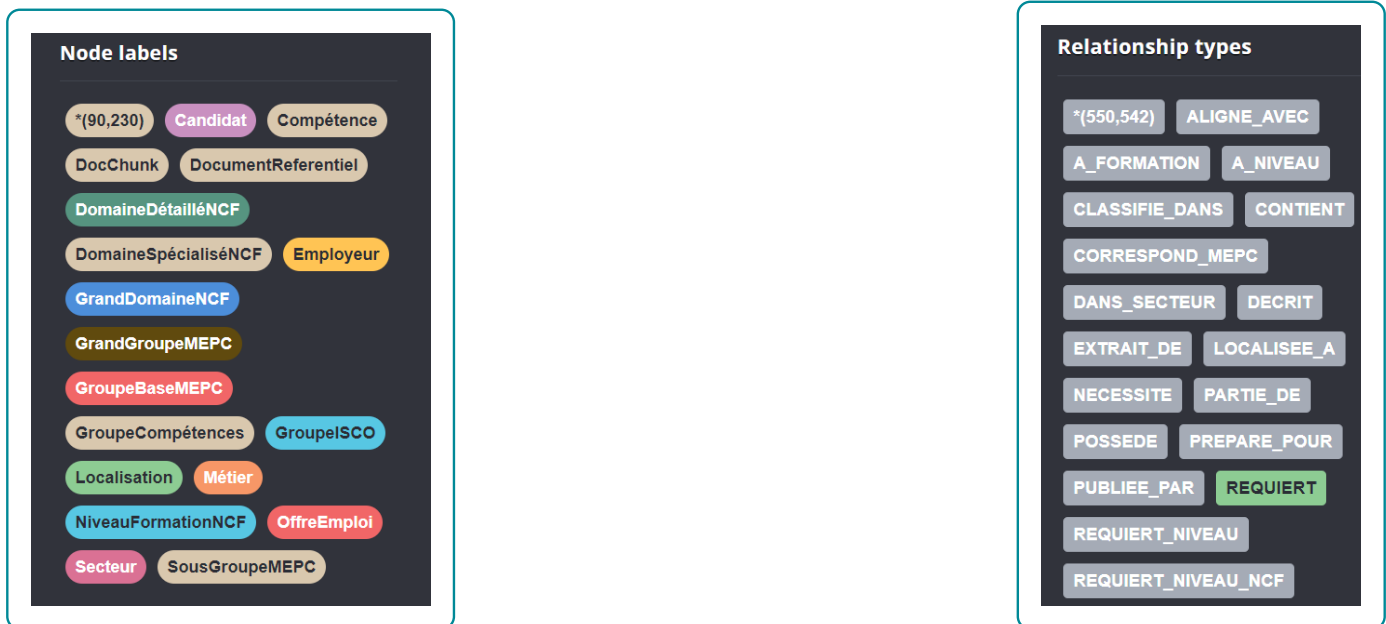
3.4.1 Schéma, noeuds et relations mobilisés par le matching

Cette sous-section décrit la partie du graphe réellement utile au matching. L'objectif n'est pas de présenter tous les objets stockés, mais de montrer comment une offre, un candidat et des référentiels deviennent interrogeables par relations.

Figure 3.9 – Schéma logique du graphe de connaissances

Source : Nos travaux, diagnostics Neo4j

La figure 3.9 montre deux fonctions du graphe : relier les candidats aux offres par les compétences et rattacher ces objets aux référentiels métier et formation. pgvector repère une proximité textuelle ; Neo4j indique par quelles relations cette proximité peut être expliquée.

Figure 3.10 – Lecture conjointe des noeuds et relations Neo4j mobilisés par le matching

(a) Types de noeuds

(b) Types de relations

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

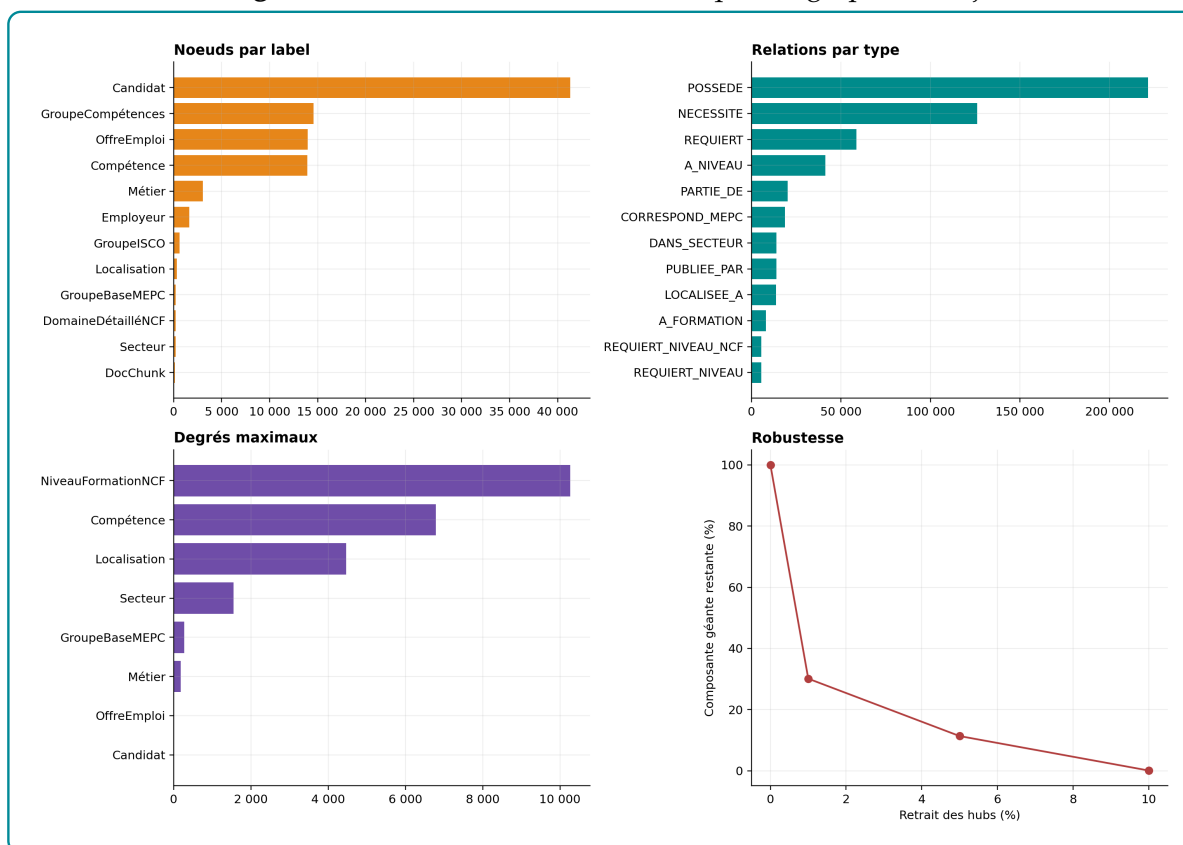
Le chemin principal est Candidat → POSSEDE → Compétence ← REQUIERT ← OffreEmploi. Il permet de distinguer les compétences couvertes des compétences manquantes. Les liens vers le métier, le niveau NCF, le secteur et la localisation complètent l'explication. Leur fiabilité

reste conditionnée par la qualité de l'extraction et de la normalisation.

3.4.2 Structure du graphe et dépendance aux hubs

Cette sous-section contrôle si le graphe est exploitable pour le GraphRAG. Dans le workflow, ce diagnostic précède le score hybride : si le graphe est trop fragmenté, Neo4j ne peut pas expliquer les recommandations ; s'il dépend trop de quelques noeuds génériques, il peut au contraire produire des correspondances trop larges.

Figure 3.11 – Tableau de bord statistique du graphe Neo4j



Source : Nos travaux, diagnostics Neo4j

La figure 3.11 regroupe quatre diagnostics complémentaires. Le panneau des labels montre que le graphe est dominé par les candidats (41 298 noeuds), puis par les groupes de compétences, les offres (13 957) et les compétences (13 939). Cette composition est cohérente avec le workflow : relier des profils nombreux à un vivier d'offres à travers des compétences normalisées.

Le deuxième panneau, « Relations par type », indique que les relations POSSEDE dominent avec 221 413 liens, suivies par NECESSITE (126 051) et REQUIERT (58 519). Le graphe est donc bien centré sur la chaîne candidat-compétence-offre, avec un enrichissement par les métiers

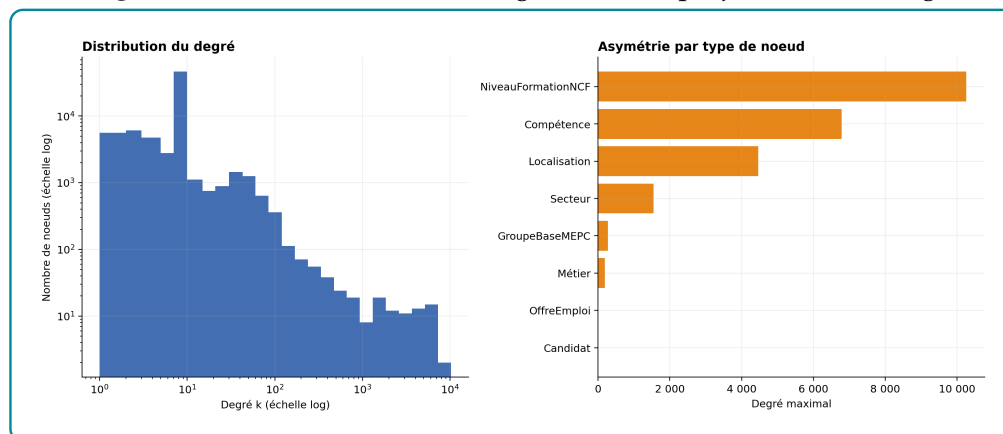
de référence. C'est utile pour expliquer le *skill gap*, mais cela rend aussi le système sensible à la qualité d'extraction des compétences : une compétence mal extraite crée directement un lien erroné.

Le troisième panneau, « Degrés maximaux », révèle la présence de hubs. Les niveaux NCF atteignent 10 261 connexions et certaines compétences 6 787, alors que le degré médian d'une compétence est de 4. Ces hubs sont utiles pour connecter le graphe, mais ils sont peu discriminants pour recommander une offre précise. Ils ne doivent donc pas être interprétés comme des preuves fortes de compatibilité.

Le quatrième panneau, « Robustesse », teste la dépendance du graphe à ces hubs. Lorsque les 1 % de noeuds les plus connectés sont retirés, la composante géante ne conserve plus que 30,1 % de sa taille initiale ; après retrait des 5 %, elle tombe à 11,3 % ; après 10 %, elle s'effondre presque totalement. Le résultat est clair : la connectivité du graphe repose fortement sur quelques noeuds génériques. Cette faiblesse ne bloque pas l'usage de Neo4j, mais elle impose de ne pas utiliser le degré brut comme score de pertinence.

Le graphe contient au total 90 230 noeuds et 550 542 relations. Sa faible densité ($6,76 \times 10^{-5}$) n'indique donc pas un manque d'information ; elle traduit un schéma spécialisé où une offre est reliée aux entités qui l'expliquent, et non à toutes les entités disponibles. La projection de matching regroupe 72 516 noeuds et 499 080 arêtes dans une composante connexe : les chemins candidat-compétence-offre existent, mais leur pertinence dépend de la qualité des relations POSSEDE et REQUIERT. La composition détaillée est reportée en annexe.

Figure 3.12 – Distribution des degrés dans la projection matching



Source : Nos travaux, diagnostics Neo4j sur la projection des relations utilisées pour le matching.

La distribution est asymétrique : les niveaux NCF atteignent 10 261 connexions et certaines compétences 6 787, alors que le degré médian d'une compétence est de 4. Quelques libellés

généraux concentrent donc une grande partie des chemins et sont peu discriminants.

Le score Neo4j utilise donc la proportion de compétences requises couvertes, et non le degré brut des noeuds. Une compétence fréquente n'améliore pas automatiquement une recommandation ; elle ne compte que si elle est exigée par l'offre et associée au candidat. Cette règle protège le reranking contre le bruit de popularité et prépare la section suivante, où Neo4j devient un signal pondéré du score hybride.

3.5 Score hybride et reranking des recommandations

Le moteur combine le rappel sémantique de pgvector et la couverture de compétences calculée dans Neo4j. Il réordonne les offres avant leur transmission au module de génération.

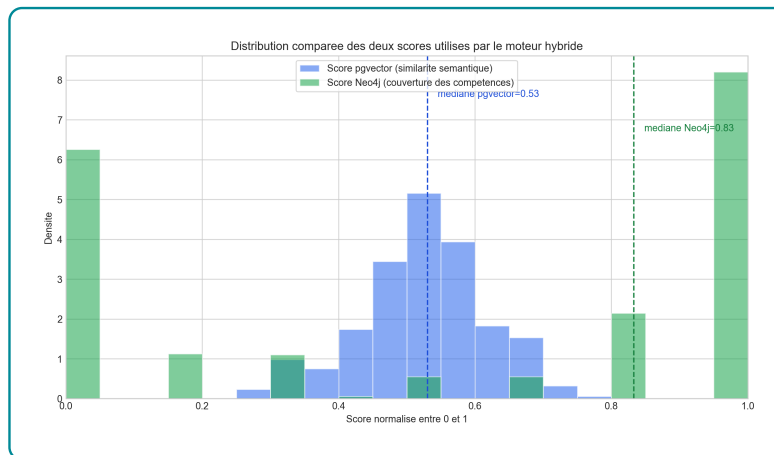
3.5.1 Formalisation du score hybride

Dans la version calibrée, le score combine deux signaux :

$$S_{\text{hybride}} = \alpha S_{\text{pgvector}} + \beta S_{\text{Neo4j}}, \quad \alpha + \beta = 1$$

où `score_sem` mesure la similarité pgvector et `taux_match` la part des compétences requises couvertes dans Neo4j. Le niveau NCF, le secteur et la localisation restent disponibles pour l'explication, mais ne reçoivent pas de poids autonome dans cette calibration.

Figure 3.13 – Distribution comparée des scores pgvector et Neo4j utilisés dans le reranking



Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings

Sur 690 couples candidat-offre, le score pgvector est continu et centré autour de 0,52, tandis que 31,30 % des scores Neo4j sont nuls. Le premier panneau montre donc une proximité textuelle graduelle ; le second montre un signal relationnel plus sélectif. pgvector distingue des

degrés de ressemblance entre presque tous les couples ; Neo4j écarte plus nettement ceux pour lesquels aucune compétence requise n'est couverte.

Table 3.4 – Résultats de calibration des poids du score hybride

Objectif optimisé	Outil	α Sem	β Neo4j	NDCG@10
NDCG@10	Neo4j seul	0,0000	1,0000	0,9697
(NDCG@10 + Recall@10 + Precision@10)/3	Neo4j seul	0,0000	1,0000	0,9697
(MRR@10 + Precision@10)/2	Mixte Optuna	0,6615	0,3385	0,9674

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

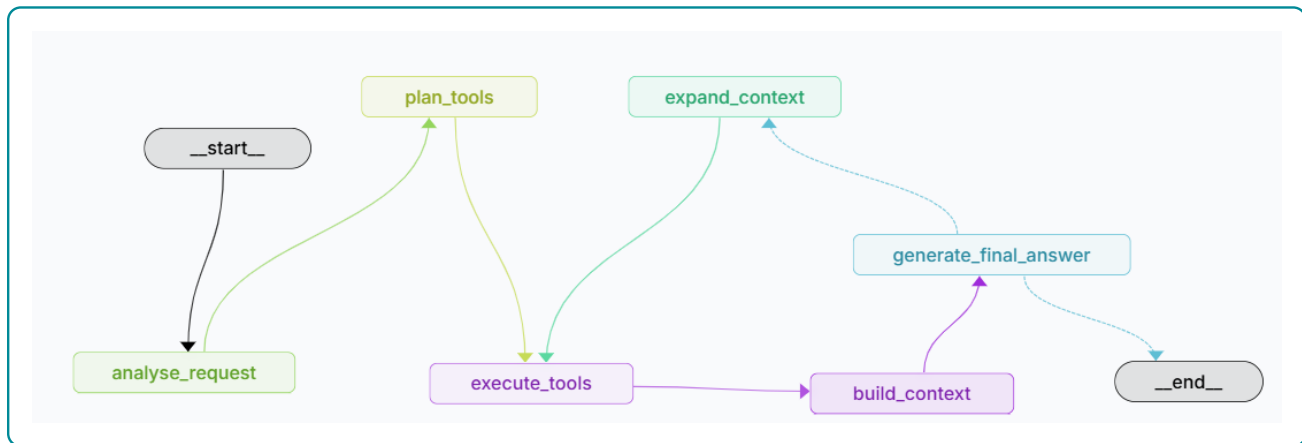
Avec 500 essais Optuna, l'objectif NDCG@10 retient $\alpha = 0$ et $\beta = 1$: dans le vivier déjà présélectionné par pgvector, la couverture de compétences ordonne le mieux l'ensemble du top 10. L'objectif MRR@10–Precision@10 retient plutôt 0,6615 pour pgvector et 0,3385 pour Neo4j, car il valorise davantage le premier résultat pertinent. Les poids sont donc propres à l'objectif et à l'échantillon de calibration.

3.5.2 Effet du reranking hybride

Neo4j ne relance pas la recherche : il réordonne le vivier pgvector selon la couverture de compétences. Les déplacements de rang et l'ablation avec ou sans graphe sont analysés au chapitre 4. Le tableau A.12 en annexe conserve le résumé numérique.

3.6 Orchestration du système par LangGraph

Cette dernière brique relie les composants précédents dans une chaîne exécutable. LangGraph conserve un état partagé contenant la requête, le cas d'usage, les outils planifiés, leurs résultats, le contexte construit, la réponse, la décision du critic et les traces d'exécution. L'orchestrateur ne calcule donc pas lui-même la recommandation : il détermine quelles briques appeler et dans quel ordre.

Figure 3.14 – Workflow agentique orchestré dans LangGraph Studio

Source : Nos travaux, visualisation LangGraph Studio du workflow implémenté.

La figure montre le parcours complet de la requête : analyse, planification des outils, exécution, construction du contexte, génération, puis fin ou révision. Les flèches de retour vers `expand_context` indiquent que le système peut enrichir les preuves avant de répondre. Elle situe donc LangGraph comme couche de coordination, pas comme moteur de scoring.

3.6.1 Analyse de la requête et choix des outils

Le nœud `analyse_request` extrait la question, recherche un identifiant candidat et classe la demande dans un cas d'usage : diagnostic, recommandation, analyse de *skill gap*, consultation d'un référentiel, interrogation du graphe, orientation métier, synthèse globale ou recherche générale. Le nœud `plan_tools` associe ensuite ce cas aux outils nécessaires.

Le registre contient six outils principaux. Ils couvrent le diagnostic des services, les recherches sémantique et documentaire dans pgvector, l'interrogation Neo4j par Text2Cypher, la recommandation hybride et la synthèse globale du graphe. Ce routage limite les appels inutiles : une question sur un référentiel mobilise les chunks pgvector, une question relationnelle mobilise Neo4j et une recommandation candidat-offre appelle le moteur hybride.

3.6.2 Exécution des outils et construction du contexte

`execute_tools` appelle les outils planifiés et conserve chaque résultat ou erreur dans l'état. `build_context` rassemble ensuite les sorties hétérogènes dans une structure commune : offres candidates, compétences couvertes ou manquantes, lignes Neo4j, résultats sémantiques, fragments documentaires et état des services.

Cette normalisation sépare la récupération de la génération. Le LLM ne consulte pas directement les bases et ne choisit pas librement les données à utiliser ; il reçoit un contexte déjà

récupéré et structuré. Les erreurs d'un outil restent également traçables sans supprimer les résultats valides produits par les autres outils.

3.6.3 Génération contrôlée et boucle de révision

`generate_final_answer` produit une réponse française à partir du contexte et impose la citation des identifiants ou sources disponibles. Si OpenRouter est indisponible, le système restitue les données récupérées sous une forme structurée plutôt que d'inventer une réponse.

Le `critic` évalue ensuite l'ancrage de la réponse. Une décision `accept` termine le workflow. Une décision `revise` active `expand_context`, augmente le top- K dans la limite de 10 et ajoute les recherches pgvector ou Neo4j qui n'avaient pas encore été exécutées. Le contexte est alors reconstruit et la réponse régénérée. Cette boucle est limitée à une seule révision afin d'éviter une exécution indéfinie ; le chapitre 4 présente la calibration du seuil utilisé par ce contrôle.

Conclusion

Le chapitre a matérialisé la chaîne suivante : données normalisées, embeddings spécialisés, rappel pgvector, contrôle Neo4j, reranking hybride et orchestration LangGraph. Chaque brique remplit une fonction distincte, tandis que l'orchestrateur assure leur enchaînement et la traçabilité des décisions. Le résultat reste dépendant de la qualité des données, des relations de compétences et du contexte récupéré ; le chapitre 4 mesure l'effet de ces choix sur le classement et la génération.

Évaluation de la performance du système

Ce chapitre évalue le système implémenté au chapitre précédent. Il ne répète pas la description technique des briques ; il vérifie ce que ces briques apportent effectivement au fonctionnement du système. L'évaluation suit donc une logique en cinq niveaux : qualité du retrieval sémantique, effet du graphe sur le classement, validité fonctionnelle des requêtes GraphRAG, calibration du score hybride et contrôle de la génération finale par le critic.

Deux précautions guident l'interprétation. Premièrement, l'évaluation du classement est conduite sans génération LLM : elle mesure la qualité du retrieval et du reranking, non la qualité rédactionnelle de la réponse. Deuxièmement, la pertinence utilisée dans l'ablation est le proxy défini dans la méthodologie à partir du recouvrement lexical, de l'alignement sectoriel et de la compatibilité NCF. Sa valeur continue sert de gain pour le NDCG ; sa version binarisée par le seuil $\max(0,35, Q_{0,70})$ sert à Precision, Recall et MRR. Ce label faible permet de comparer les variantes du système, mais ne remplace pas une annotation humaine par des recruteurs ou des conseillers métier.

4.1 Protocole expérimental et métriques retenues

L'évaluation mobilise trois familles d'artefacts. Le benchmark du modèle d'embeddings utilise le jeu de test issu des paires contrastives offre–profil, soit 473 requêtes. L'ablation pgvector contre pgvector enrichi par Neo4j utilise un tirage de 80 candidats avec graine fixée à 42 ; 69 candidats sont effectivement évalués après contrôles d'exécution. Enfin, la calibration du critic utilise un jeu contrastif de 102 évaluations construit à partir de 51 cas réels et de leurs contextes mélangés.

Les métriques de retrieval retenues sont classiques en recherche d'information : Recall@K, Precision@K, NDCG@K, MRR@K et HitRate@K. Le Recall@K mesure si les éléments pertinents sont récupérés dans les premiers résultats. Le NDCG@K évalue aussi leur ordre. Le MRR@K met l'accent sur le rang de la première bonne recommandation. Ces métriques sont complémentaires : un système peut retrouver une bonne offre dans le top 10 tout en la plaçant trop bas pour être utile à l'utilisateur.

Pour l'ablation, les deux configurations reçoivent le même pool initial de 30 offres produit par pgvector. La configuration `pgvector` seul conserve l'ordre vectoriel. La configuration

`pgvector` + graphe enrichit ce même pool avec Neo4j, puis le rerange à partir des relations candidat-offre-compétence. Cette construction isole l'effet du graphe : la différence observée ne vient pas d'un corpus différent, mais de l'information relationnelle ajoutée au classement.

4.2 Retrieval sémantique et apport mesuré du graphe

4.2.1 Modèle d'embeddings spécialisé

Le premier test vérifie si le fine-tuning du SentenceTransformer améliore réellement le retrieval. Le tableau 4.1 compare le modèle spécialisé à plusieurs modèles généralistes ou multilingues sur les mêmes 473 requêtes de test.

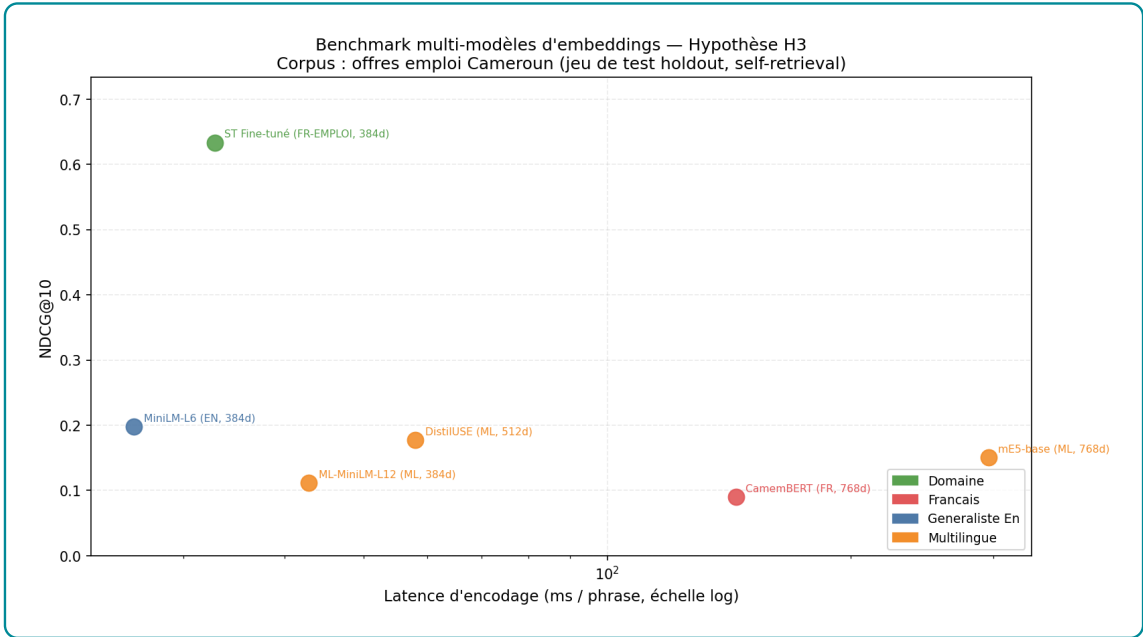
Table 4.1 – Benchmark des modèles d'embeddings sur 473 requêtes de test

Modèle	NDCG@10	MRR@10	Recall@10	Latence ms/phrased
ST fine-tuné emploi-compétences	0.6336	0.5653	0.8520	32.77
MiniLM-L6 généraliste	0.1980	0.1603	0.3192	26.03
DistilUSE multilingue	0.1773	0.1434	0.2896	57.96
mE5-base multilingue	0.1505	0.1274	0.2262	295.76
ML-MiniLM-L12	0.1115	0.0922	0.1734	42.82
CamemBERT sentence	0.0904	0.0771	0.1332	144.15

Source : Nos travaux, évaluation SentenceTransformer sur le jeu de test offre-profil.

Le modèle fine-tuné atteint un Recall@10 de 0.8520, contre 0.3192 pour MiniLM-L6 généraliste. L'écart est important : le modèle spécialisé retrouve beaucoup plus souvent l'élément attendu dans les dix premiers résultats. Le NDCG@10 passe de 0.1980 à 0.6336, ce qui montre aussi un meilleur ordre des résultats pertinents. La latence moyenne reste contenue à 32.77 ms par phrase, ce qui reste compatible avec une recherche interactive locale.

Figure 4.1 – Comparaison des modèles d’embeddings sur le jeu de test



Source : Nos travaux, évaluation SentenceTransformer sur le jeu de test offre–profil.

La figure 4.1 confirme que le gain ne vient pas seulement d’une métrique isolée. Le modèle fine-tuné domine sur la qualité de classement, le rappel et le compromis performance–latence. Ce résultat justifie son utilisation comme encodeur principal de pgvector. La barre de rappel indique la capacité à retrouver l’offre attendue dans le top 10 ; la barre de NDCG mesure si cette offre est bien placée ; la latence vérifie que le modèle reste utilisable dans le workflow. Le bon résultat n’est donc pas seulement statistique : il est aussi opérationnel pour alimenter rapidement pgvector.

4.2.2 Ablation pgvector seul contre pgvector enrichi par Neo4j

Le deuxième test mesure l’apport du graphe. Le même pool initial de 30 offres est utilisé dans les deux configurations ; seul le reranking par les relations Neo4j change. Le tableau 4.2 présente les résultats obtenus sur les 69 candidats effectivement évalués.

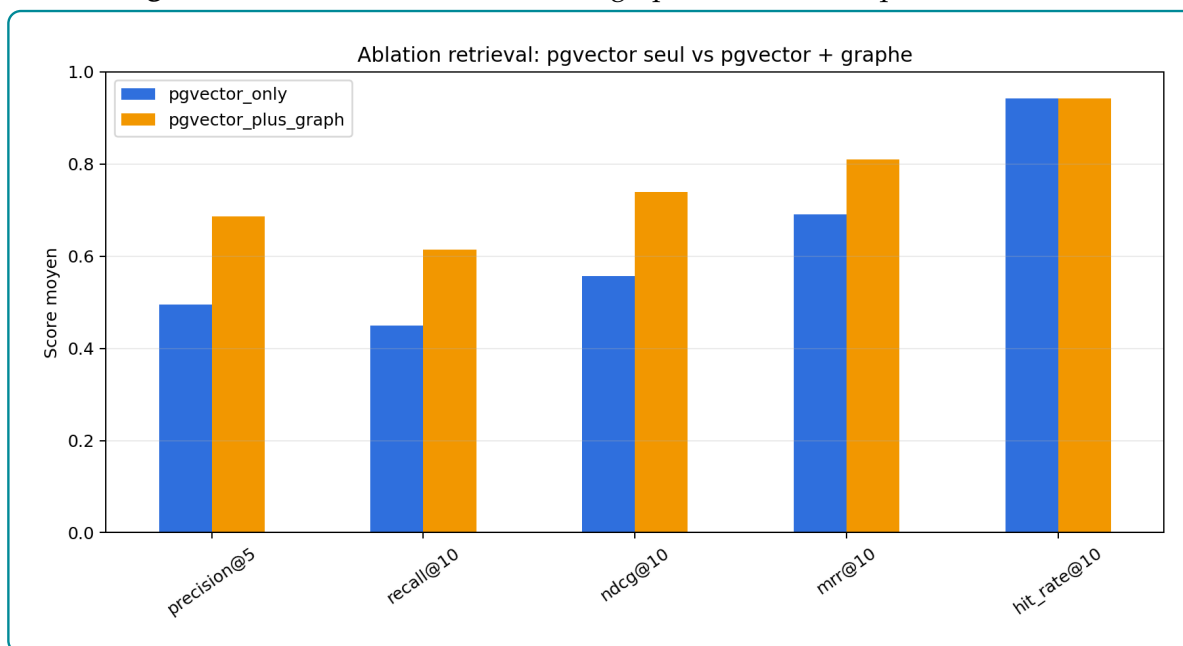
Table 4.2 – Comparaison retrieval : pgvector seul versus pgvector + graphe

Configuration	Precision@5	Recall@10	NDCG@10	MRR@10	HitRate@10
pgvector seul	0.4957	0.4491	0.5575	0.6908	0.9420
pgvector + graphe	0.6870	0.6147	0.7392	0.8093	0.9420
Différence	+0.1913	+0.1656	+0.1817	+0.1185	0.0000

Source : Nos travaux, ablation du moteur de recommandation sur 69 candidats évalués.

L'ajout de Neo4j améliore Precision@5, Recall@10, NDCG@10 et MRR@10. Le Recall@10 passe de 0.4491 à 0.6147, soit un gain de 0.1656. Le NDCG@10 passe de 0.5575 à 0.7392, ce qui montre que le graphe ne se contente pas de retrouver plus d'offres jugées pertinentes : il les place aussi dans un meilleur ordre. Le HitRate@10 reste stable à 0.9420 ; le pool vectoriel contient donc souvent au moins une bonne offre, mais Neo4j améliore son rang.

Figure 4.2 – Effet de l'enrichissement graphe sur les métriques de retrieval



Source : Nos travaux, ablation du moteur de recommandation sur 69 candidats évalués.

La lecture correcte est séquentielle. pgvector reste le premier étage : il constitue rapidement un vivier d'offres plausibles. Neo4j intervient ensuite pour départager ce vivier à partir des relations POSSEDE, REQUIERT, A_NIVEAU et REQUIERT_NIVEAU. L'amélioration mesurée vient donc du reranking relationnel, pas d'un remplacement de la recherche vectorielle. La figure met en évidence trois effets. Precision@5 augmente, ce qui signifie que le haut de liste devient plus utile. Recall@10 augmente, donc davantage d'offres pertinentes sont récupérées dans les dix premières positions. NDCG@10 et MRR@10 progressent aussi, ce qui montre que les bonnes offres ne sont pas seulement présentes : elles sont mieux ordonnées.

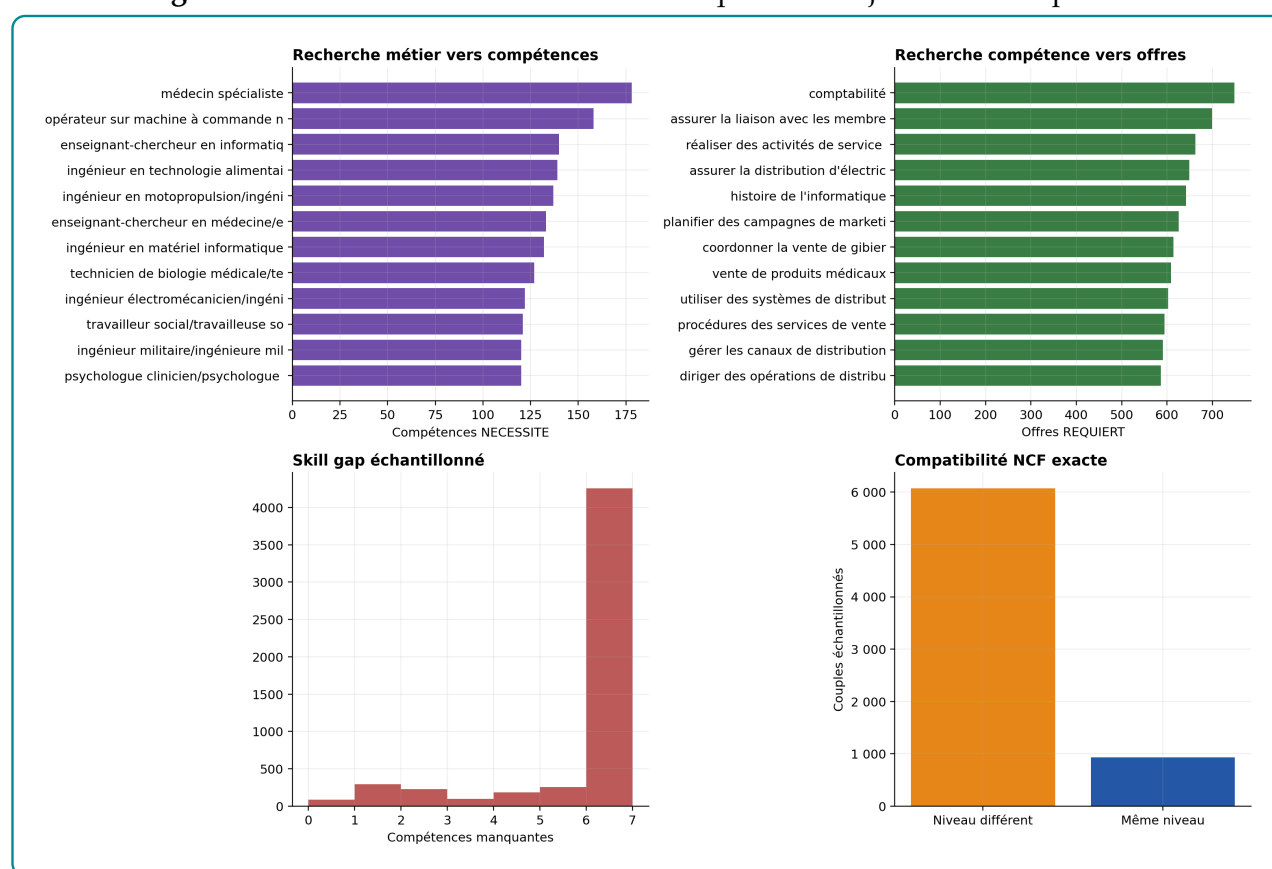
4.3 Validation fonctionnelle du GraphRAG

L'ablation précédente mesure l'effet du graphe sur le classement, mais elle ne suffit pas à vérifier que le GraphRAG fonctionne correctement dans les usages métier. Cette section évalue donc la chaîne fonctionnelle : partir d'une demande utilisateur, activer les chemins Neo4j utiles, récupérer les entités reliées, puis transformer ces relations en contexte exploitable par la

génération. La figure 4.3 synthétise ces contrôles selon cinq angles complémentaires : recherche métier vers compétences, recherche compétence vers offres, calcul de *skill gap*, compatibilité NCF et comparaison RAG contre GraphRAG.

Le rôle de cette figure n'est pas seulement illustratif. Elle sert de tableau de bord fonctionnel : chaque barre correspond à une capacité attendue du système. Si une barre est faible, le problème ne vient pas nécessairement du LLM ; il peut venir d'une relation manquante, d'un référentiel incomplet ou d'un chemin Neo4j trop pauvre. La lecture des sous-sections suivantes reprend donc les mêmes familles de tests que la figure afin de relier les résultats chiffrés au fonctionnement concret du GraphRAG.

Figure 4.3 – Évaluation fonctionnelle des requêtes Neo4j utiles au GraphRAG



Source : Nos travaux, diagnostics Neo4j et GraphRAG sur les scénarios fonctionnels du système.

La figure doit être lue comme un contrôle de couverture fonctionnelle. Les deux premiers blocs vérifient la navigation relationnelle métier-compétence-offre ; les blocs centraux contrôlent le *skill gap* et le niveau NCF ; le dernier bloc compare le classement obtenu avec et sans graphe. Elle confirme donc que Neo4j sert à la fois à récupérer des relations, à expliquer les écarts et à améliorer le reranking.

4.3.1 Requêtes relationnelles métier, compétence et offre

La première partie de la figure 4.3 vérifie la navigation depuis un métier vers les compétences attendues. Le graphe part d'un noeud **Métier** et suit la relation **NECESSITE** pour retrouver les compétences associées. Les métiers les plus riches sont très connectés : « médecin spécialiste » est relié à 178 compétences, « opérateur sur machine à commande numérique » à 158 et « enseignant-chercheur en informatique » à 140. Cette richesse est utile pour l'orientation, car elle donne au GraphRAG un contexte métier large. Elle ne doit cependant pas être lue comme une liste minimale d'exigences pour chaque offre : une offre précise ne mobilise qu'une partie de ce contexte.

La deuxième partie de la même figure contrôle le sens inverse : à partir d'une compétence, le graphe retrouve les offres qui la requièrent. La compétence « comptabilité » est reliée à 748 offres, « réaliser des activités de service après-vente » à 662 offres et « planifier des campagnes de marketing sur les médias sociaux » à 626 offres. Ce résultat confirme que Neo4j sert bien de moteur de recherche relationnelle pour compléter pgvector. Il révèle aussi une limite importante : certaines compétences fréquentes deviennent des hubs peu discriminants. Dans le score final, elles doivent donc être pondérées avec prudence, sinon le système risque de favoriser des correspondances trop générales.

4.3.2 Skill gap, niveau NCF et comparaison GraphRAG

La troisième lecture de la figure 4.3 porte sur le *skill gap*. Le calcul compare les compétences **REQUIERT** par l'offre aux compétences **POSSEDE** par le candidat. Sur l'échantillon fonctionnel, le déficit moyen est de 5,30 compétences et la médiane vaut 6. Ce résultat est utile parce qu'il montre que le GraphRAG ne se limite pas à dire qu'un candidat correspond ou ne correspond pas. Il identifie les compétences manquantes et peut transformer un mauvais appariement brut en recommandation explicable de montée en compétences.

La quatrième lecture concerne la compatibilité NCF. Elle vérifie si le niveau du candidat correspond au niveau requis par l'offre. Dans l'échantillon évalué, 13,29 % des couples partagent exactement le même niveau. Ce taux confirme que le niveau de formation apporte un signal structurant, mais il ne doit pas devenir un couperet automatique. Selon l'expérience et les compétences détenues, un écart de niveau peut rester acceptable ; le graphe doit donc éclairer la décision, pas la remplacer.

Enfin, la dernière partie de la figure 4.3 met en relation ces contrôles fonctionnels avec la comparaison RAG contre GraphRAG. Le RAG vectoriel produit le pool initial, tandis que le

GraphRAG ajoute les relations Neo4j pour reranger les offres et fournir des explications. Les résultats sont nets : le NDCG@10 passe de 0.5575 à 0.7392, le Recall@10 de 0.4491 à 0.6147 et le MRR@10 de 0.6908 à 0.8093. Le GraphRAG améliore donc l'ordre des recommandations et pas seulement leur justification textuelle.

4.4 Reranking hybride et calibration du score

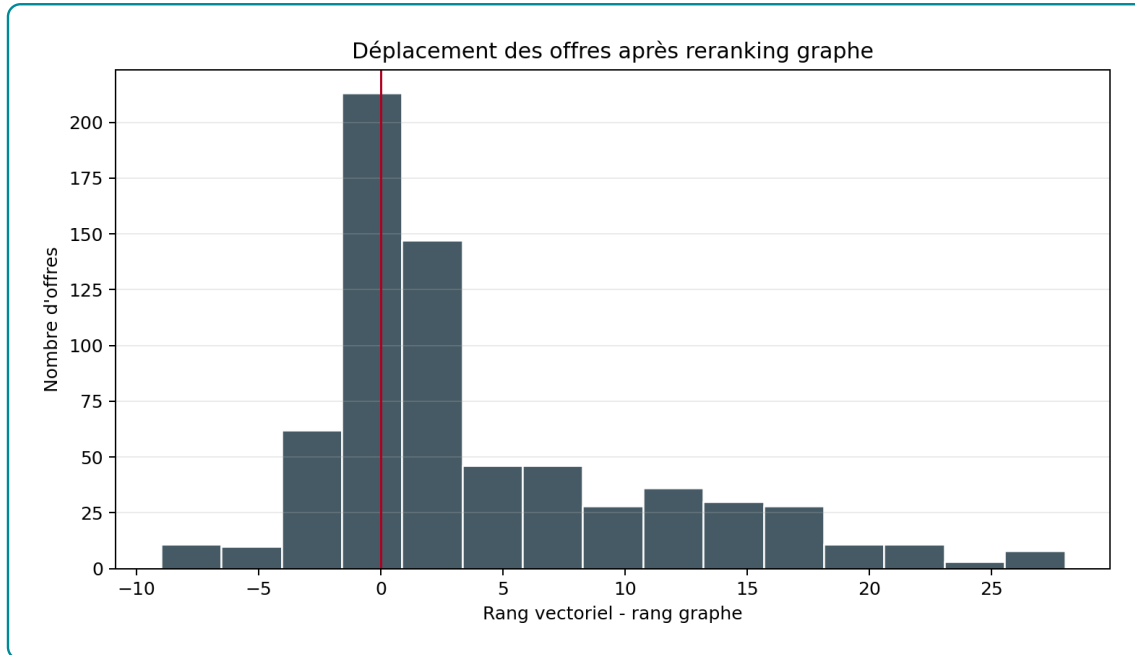
4.4.1 Déplacements de rang après reranking par le graphe

Le déplacement de rang est calculé par la différence

$$\Delta r = r_{\text{pgvector}} - r_{\text{hybride}}.$$

Une valeur positive indique qu'une offre remonte après l'ajout du score Neo4j ; une valeur négative indique une rétrogradation ; une valeur nulle signifie que son rang ne change pas. Dans cette expérience, le score Neo4j correspond au taux de couverture des compétences requises par l'offre. Le graphique ne mesure donc pas directement l'effet du secteur ou du niveau NCF.

Figure 4.4 – Distribution des déplacements de rang après reranking par le graphe



Source : Nos travaux, ablation du moteur hybride et analyse des déplacements de rang.

Sur les 690 couples candidat-offre associés aux 69 candidats évalués, 59,86 % des offres remontent, 13,77 % reculent et 26,38 % conservent leur position. Le rang moyen passe de 9,51

avec pgvector seul à 5,50 après reranking, soit un gain moyen de 4,01 positions. Le déplacement médian n'est toutefois que de +1 : l'amélioration moyenne est donc influencée par un nombre plus limité de remontées importantes et ne signifie pas que chaque offre gagne quatre places.

La figure montre ainsi que Neo4j modifie réellement l'ordre du vivier initial, principalement en favorisant les offres dont les compétences requises sont mieux couvertes. Une rétrogradation n'est pas nécessairement une erreur : elle peut signaler qu'une proximité textuelle élevée n'est pas confirmée par les relations de compétences. Inversement, le résultat dépend de la qualité des liens [POSSEDE](#) et [REQUIERT](#). Cette analyse démontre un effet mesurable du graphe sur le classement, mais pas encore une pertinence métier définitive sans annotation humaine.

4.4.2 Pondérations optimales avec Optuna

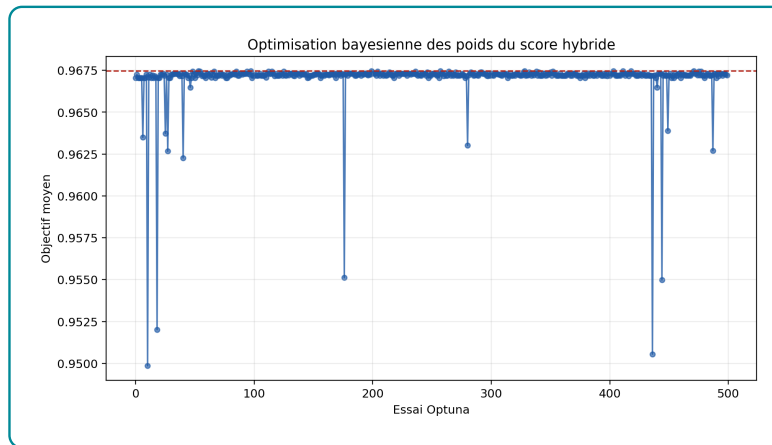
Afin de ne pas conserver des pondérations arbitraires, les poids du score hybride ont été optimisés avec Optuna sur 690 couples candidat-offre issus des 69 candidats évalués. Trois fonctions objectif sont comparées : NDCG@10, moyenne NDCG@10–Recall@10–Precision@10 et moyenne MRR@10–Precision@10. La pertinence utilisée est [proxy_relevance](#), un label faible construit à partir des métadonnées normalisées.

Table 4.3 – Pondérations retenues selon la fonction objectif du score hybride

Fonction objectif	w_{sem}	w_{Neo4j}	NDCG@10	Precision@10	Recall@10	MRR@10
NDCG@10	0.0000	1.0000	0.9697	0.5000	1.0000	0.8109
NDCG@10 + Recall@10 + Precision@10	0.0000	1.0000	0.9697	0.5000	1.0000	0.8109
MRR@10 + Precision@10	0.6615	0.3385	0.9674	0.5000	1.0000	0.8183

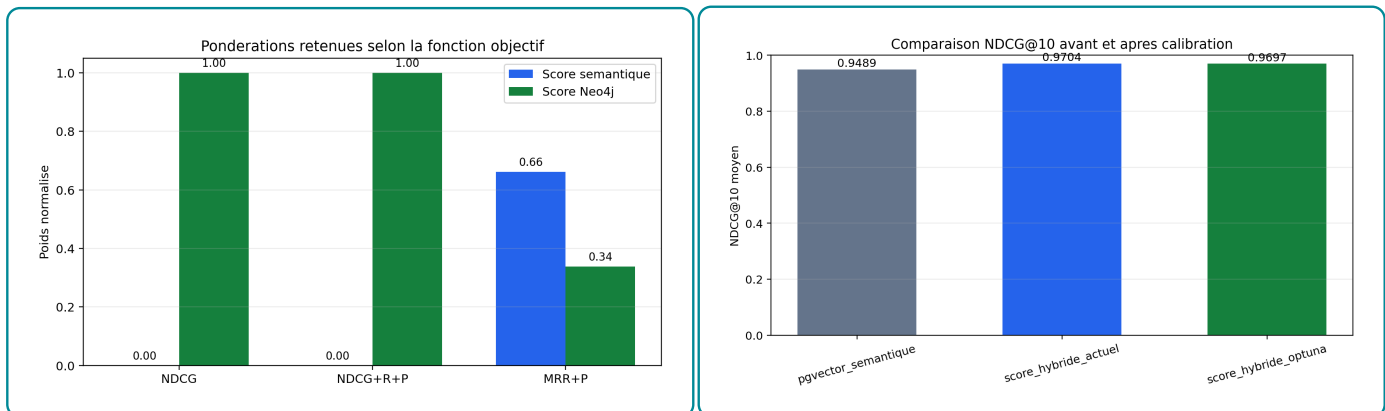
Source : Nos travaux, optimisation Optuna des poids du score hybride sur les sorties d'ablation.

Le résultat est clair mais doit être bien défendu. Lorsque l'objectif privilégie l'ordre global du top 10, Optuna retient un score Neo4j pur : $w_{sem} = 0$ et $w_{Neo4j} = 1$. Cela ne signifie pas que pgvector est inutile ; cela signifie qu'une fois le pool construit par pgvector, le meilleur signal de reclassement dans ce protocole est la couverture relationnelle calculée dans Neo4j. Lorsque l'objectif privilégie davantage la première bonne recommandation, le compromis devient hybride avec $w_{sem} = 0,6615$ et $w_{Neo4j} = 0,3385$.

Figure 4.5 – Évolution de l'objectif NDCG@10 au cours des essais Optuna

Source : Nos travaux, optimisation Optuna des poids du score hybride.

Cette figure montre la recherche menée par Optuna. Chaque essai correspond à une pondération candidate entre pgvector et Neo4j ; la valeur affichée est le NDCG@10 obtenu après reranking. La stabilisation des meilleurs essais indique que l'optimisation converge vers une zone de poids favorable, mais elle ne garantit pas que ces poids resteraient optimaux sur un jeu annoté par des recruteurs.

Figure 4.6 – Comparaison des pondérations retenues selon la fonction objectif

Source : Nos travaux, optimisation Optuna des poids du score hybride.

La figure compare deux lectures. Le panneau multi-objectifs montre que les poids changent selon la priorité retenue : qualité globale du top 10 ou première bonne recommandation. Le panneau centré sur NDCG@10 montre au contraire que, pour ordonner l'ensemble du top 10 dans ce protocole, le signal Neo4j domine. Cette différence justifie de présenter les poids comme dépendants de l'objectif, et non comme des coefficients universels.

La conclusion opérationnelle est la suivante : pgvector doit être conservé pour le rappel initial, tandis que Neo4j doit peser fortement dans le classement final lorsque les relations

POSSEDE et REQUIERT sont disponibles. La calibration reste cependant dépendante du proxy de pertinence ; une validation humaine est nécessaire avant d'en faire des poids définitifs.

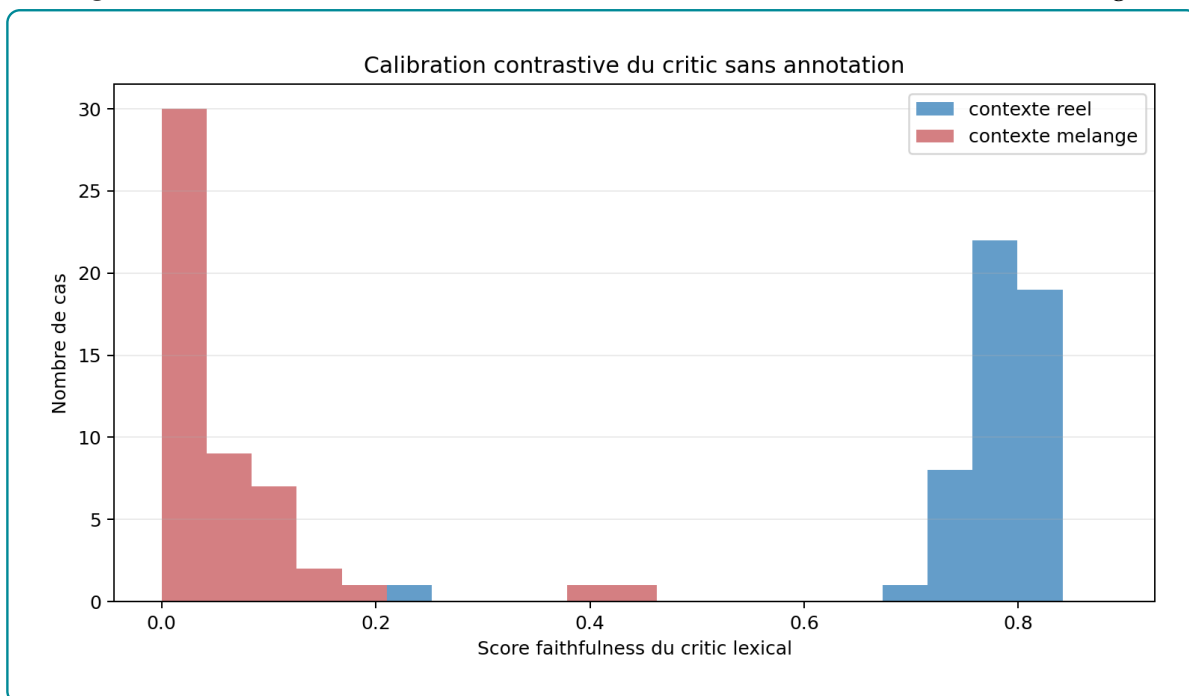
4.5 Contrôle de génération par critic

4.5.1 Calibration contrastive du seuil de fidélité

Après le retrieval et le reranking, le LLM OpenRouter formule la réponse finale. Le critic vérifie ensuite si cette réponse reste suffisamment appuyée par le contexte récupéré. Le score utilisé est un score de fidélité lexicale : il compare les mots significatifs de la réponse aux mots présents dans les preuves. Il ne juge pas la qualité métier complète de la réponse ; il vérifie seulement son ancrage dans le contexte.

Aucun jeu d'annotations humaines n'étant disponible, le seuil est calibré par contraste. Une réponse comparée à son vrai contexte doit obtenir un score plus élevé que la même réponse comparée à un contexte mélangé provenant d'une autre requête. Le protocole contient 51 cas réels évalués deux fois, soit 102 évaluations. Le score moyen vaut 0.7725 avec contexte réel contre 0.0532 avec contexte mélangé ; la médiane passe de 0.7857 à 0.0000.

Figure 4.7 – Distribution des scores du critic avec contexte réel et contexte mélangé



Source : Nos travaux, évaluation du critic à partir des réponses générées et des contextes récupérés.

La figure oppose deux distributions. Les scores avec vrai contexte sont concentrés vers des valeurs élevées, tandis que les scores avec contexte mélangé sont proches de zéro. Cette

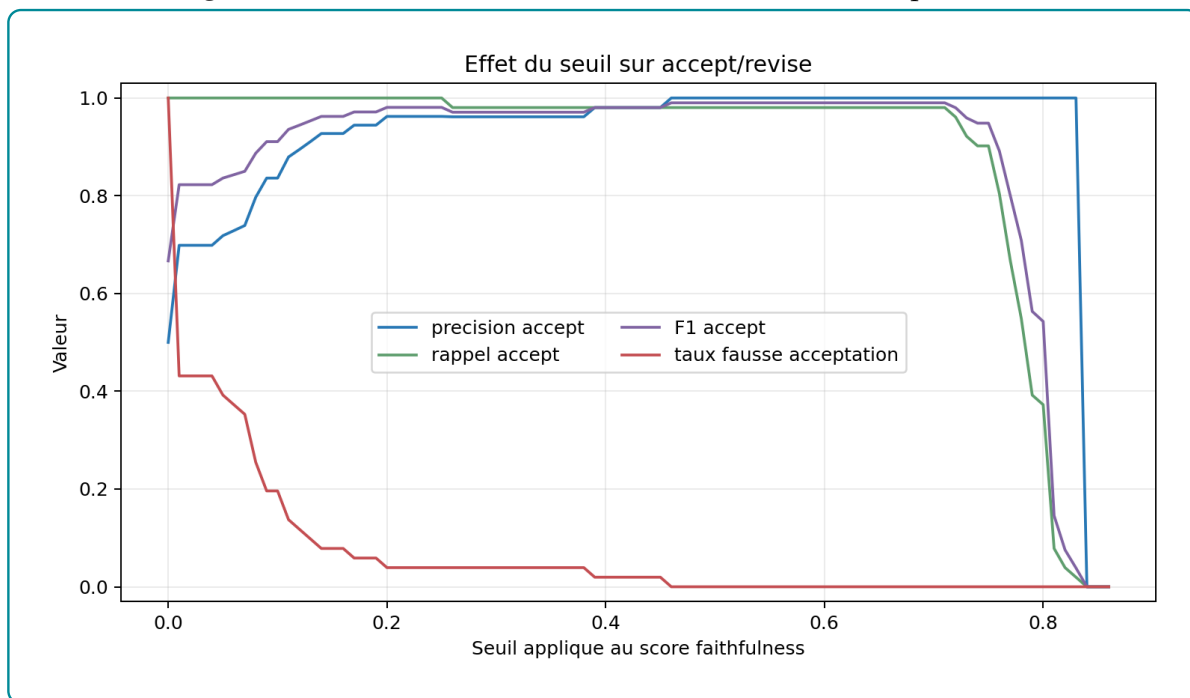
séparation montre que le critic détecte bien l’ancrage lexical dans les preuves. La limite est importante : il mesure la fidélité au contexte, pas la vérité métier complète de la réponse.

4.5.2 Interprétation des décisions accept/revise

Le seuil opérationnel retenu est 0,46. Il a été obtenu par une calibration empirique sur une grille de seuils appliquée aux 102 évaluations contrastives : 51 réponses comparées à leur vrai contexte et les mêmes 51 réponses comparées à un contexte mélangé. Pour chaque seuil candidat, le critic transforme le score continu en décision binaire : *accept* si le score est supérieur ou égal au seuil, *revise* sinon. La grille permet ensuite de compter deux erreurs : les fausses acceptations, c’est-à-dire les contextes mélangés acceptés à tort, et les fausses révisions, c’est-à-dire les vrais contextes renvoyés inutilement en révision.

La valeur 0,46 est retenue parce qu’elle maximise le compromis F1 dans ce test tout en respectant une contrainte de prudence : ne laisser passer aucun contexte mélangé. À ce niveau, aucune réponse associée à un contexte mélangé n’est acceptée, tandis que 50 réponses sur 51 restent acceptées avec leur vrai contexte. Le choix n’est donc pas théorique ; il vient du point de la grille où le système bloque les réponses les moins ancrées sans devenir excessivement sévère sur les réponses correctement contextualisées.

Figure 4.8 – Effet du seuil du critic sur les décisions accept/revise



Source : Nos travaux, évaluation du critic à partir des réponses générées et des contextes récupérés.

La figure montre l’arbitrage du seuil. Un seuil trop bas accepte trop facilement des réponses

peu ancrées ; un seuil trop haut force des révisions inutiles. Le seuil 0,46 se situe dans la zone où les contextes mélangés sont rejetés tout en conservant presque toutes les réponses associées à leur vrai contexte. C'est donc un seuil de prudence opérationnelle.

Cette calibration ne transforme pas le critic en juge parfait. Une réponse acceptée n'est pas automatiquement correcte au sens métier ; elle est seulement suffisamment ancrée lexicalement dans les preuves récupérées. Pour défendre le système devant un jury, la formulation correcte est donc : le seuil 0,46 est un seuil opérationnel calibré empiriquement par contraste, en attendant une validation humaine de la qualité réelle des recommandations générées.

Conclusion

L'évaluation montre trois résultats principaux. Premièrement, le fine-tuning du modèle d'embeddings améliore fortement le retrieval sémantique, avec un Recall@10 de 0.8520 et un NDCG@10 de 0.6336. Deuxièmement, l'ajout de Neo4j améliore le classement des offres dans le pool pgvector : le NDCG@10 passe de 0.5575 à 0.7392 et le MRR@10 de 0.6908 à 0.8093. Troisièmement, le critic distingue correctement les réponses ancrées dans leur contexte des réponses confrontées à un contexte mélangé, ce qui justifie le seuil opérationnel de 0,46.

La limite principale reste l'absence d'annotation humaine indépendante. Les résultats démontrent la cohérence interne du système et l'intérêt mesurable du graphe dans le protocole actuel. Ils ne prouvent pas encore la satisfaction d'un recruteur, d'un candidat ou d'un conseiller d'orientation. Cette limite doit être assumée : le système est techniquement validé sur ses sorties internes, mais son évaluation métier finale devra passer par un jeu annoté et des tests utilisateurs.

Conclusion générale

Ce mémoire avait pour objectif de concevoir et d'évaluer un système de recommandation hybride pour le rapprochement entre offres d'emploi, profils candidats, compétences et référentiels métier. Le choix retenu n'a pas été de confier toute la décision à un modèle génératif, mais de construire une chaîne contrôlée : nettoyage des données, adaptation des embeddings au domaine emploi-compétences, indexation vectorielle avec pgvector, structuration relationnelle dans Neo4j, reranking hybride, puis génération augmentée par un workflow agentique.

Les résultats obtenus montrent que chaque brique joue un rôle distinct. Le modèle d'embeddings améliore la récupération des éléments proches dans l'espace sémantique. pgvector fournit une mémoire rapide pour retrouver les offres, compétences et fragments documentaires pertinents. Neo4j apporte une lecture relationnelle du matching : il permet d'exploiter les liens entre métiers, compétences, formations et référentiels, puis d'améliorer le classement initial par un score de graphe. Enfin, l'orchestration LangGraph organise ces traitements dans un workflow exploitable par l'application, depuis l'analyse de la requête jusqu'à la génération de la réponse finale.

L'évaluation confirme l'intérêt de cette architecture hybride. Le fine-tuning du modèle d'embeddings améliore le retrieval, l'ajout du graphe renforce le classement des offres et le critic permet de filtrer les réponses insuffisamment ancrées dans les preuves récupérées. Le seuil opérationnel de 0,46 n'est donc pas présenté comme une vérité théorique : il correspond à une calibration empirique issue des sorties contrastives du système. Cette précision est importante, car elle évite de surinterpréter le critic comme un juge métier autonome.

La principale limite du travail reste l'absence d'une validation humaine indépendante à grande échelle. Les métriques internes démontrent la cohérence technique du pipeline, mais elles ne remplacent pas l'appréciation d'un recruteur, d'un conseiller d'orientation ou d'un candidat. Les perspectives prioritaires consistent donc à constituer un jeu annoté par des experts, tester le système auprès d'utilisateurs réels, suivre les erreurs de recommandation par famille de métiers et enrichir progressivement le graphe avec des données institutionnelles mieux normalisées.

Au total, le système proposé constitue une base opérationnelle de recommandation explicable. Sa force ne réside pas seulement dans l'utilisation d'un LLM, mais dans l'articulation entre mémoire vectorielle, mémoire relationnelle, règles de ranking et contrôle de la généra-

tion. C'est cette combinaison qui rend le dispositif plus défendable qu'une approche purement textuelle ou purement générative.

Bibliographie

- [1] Francesco RICCI et al., éd. *Recommender Systems Handbook*. Springer, 2011. DOI : [10.1007/978-0-387-85820-3](https://doi.org/10.1007/978-0-387-85820-3).
- [2] PGVECTOR CONTRIBUTORS. *pgvector : Open-source Vector Similarity Search for Postgres*. 2024. URL : <https://github.com/pgvector/pgvector> (visité le 04/05/2026).
- [3] Aidan HOGAN et al. "Knowledge Graphs". In : *ACM Computing Surveys* 54.4 (2021), p. 1-37. DOI : [10.1145/3447772](https://doi.org/10.1145/3447772).
- [4] Patrick LEWIS et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474. URL : <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [5] Dale T. MORTENSEN et Christopher A. PISSARIDES. "Job Creation and Job Destruction in the Theory of Unemployment". In : *The Review of Economic Studies* 61.3 (1994), p. 397-415. DOI : [10.2307/2297896](https://doi.org/10.2307/2297896).
- [6] Christopher A. PISSARIDES. *Equilibrium Unemployment Theory*. 2^e éd. Cambridge, MA : MIT Press, 2000.
- [7] George A. AKERLOF. "The Market for "Lemons" : Quality Uncertainty and the Market Mechanism". In : *The Quarterly Journal of Economics* 84.3 (1970), p. 488-500. DOI : [10.2307/1879431](https://doi.org/10.2307/1879431).
- [8] Michael SPENCE. "Job Market Signaling". In : *The Quarterly Journal of Economics* 87.3 (1973), p. 355-374. DOI : [10.2307/1882010](https://doi.org/10.2307/1882010).
- [9] Gary S. BECKER. *Human Capital : A Theoretical and Empirical Analysis, with Special Reference to Education*. New York : Columbia University Press, 1964.
- [10] David H. AUTOR. "The "Task Approach" to Labor Markets : An Overview". In : *Journal for Labour Market Research* 46.3 (2013), p. 185-199. DOI : [10.1007/s12651-013-0128-z](https://doi.org/10.1007/s12651-013-0128-z).
- [11] Gediminas ADOMAVICIUS et Alexander TUZHILIN. "Toward the Next Generation of Recommender Systems : A Survey of the State-of-the-Art and Possible Extensions". In : *IEEE Transactions on Knowledge and Data Engineering* 17.6 (2005), p. 734-749. DOI : [10.1109/TKDE.2005.99](https://doi.org/10.1109/TKDE.2005.99).

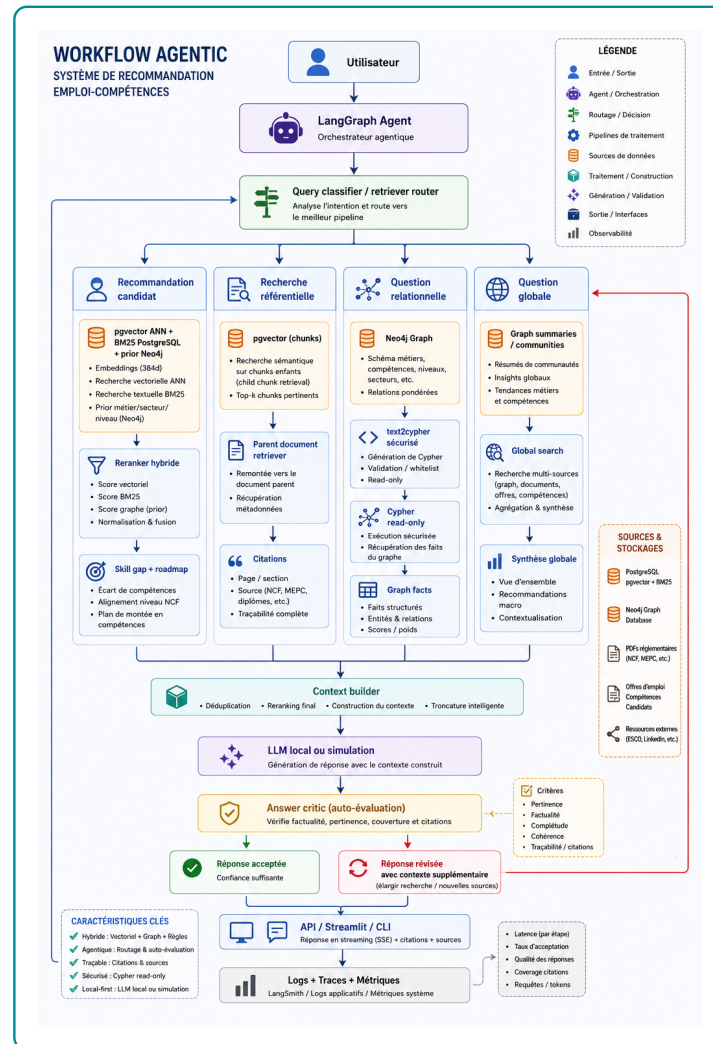
- [12] Michael J. PAZZANI et Daniel BILLSUS. "Content-Based Recommendation Systems". In : *The Adaptive Web* (2007), p. 325-341. DOI : [10.1007/978-3-540-72079-9_10](https://doi.org/10.1007/978-3-540-72079-9_10).
- [13] Yehuda KOREN, Robert BELL et Chris VOLINSKY. "Matrix Factorization Techniques for Recommender Systems". In : *Computer* 42.8 (2009), p. 30-37. DOI : [10.1109/MC.2009.263](https://doi.org/10.1109/MC.2009.263).
- [14] Xiangnan HE et al. "Neural Collaborative Filtering". In : *Proceedings of the 26th International Conference on World Wide Web*. 2017, p. 173-182.
- [15] Robin BURKE. "Hybrid Recommender Systems : Survey and Experiments". In : *User Modeling and User-Adapted Interaction* 12 (2002), p. 331-370. DOI : [10.1023/A:1021240730564](https://doi.org/10.1023/A:1021240730564).
- [16] Chen YANG et al. "Modeling Two-way Selection Preference for Person-Job Fit". In : *Proceedings of the 16th ACM Conference on Recommender Systems*. 2022, p. 102-112.
- [17] A. T. WASI. "HRGraph : Leveraging LLMs for HR Data Knowledge Graphs with Information Propagation-based Job Recommendation". In : *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models*. 2024, p. 56-62.
- [18] B. YANG et Z. SHEN. "Knowledge Graph Construction and Talent Competency Prediction for Human Resource Management". In : *Alexandria Engineering Journal* 121 (2025), p. 223-235.
- [19] Ashish VASWANI et al. "Attention Is All You Need". In : *Advances in Neural Information Processing Systems*. 2017, p. 5998-6008.
- [20] Tomas MIKOLOV et al. "Distributed Representations of Words and Phrases and their Compositionality". In : *Advances in Neural Information Processing Systems*. T. 26. 2013. URL : <https://proceedings.neurips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality>.
- [21] Dzmitry BAHDANAU, Kyunghyun CHO et Yoshua BENGIO. "Neural Machine Translation by Jointly Learning to Align and Translate". In : *International Conference on Learning Representations*. 2015. URL : <https://arxiv.org/abs/1409.0473>.
- [22] Jacob DEVLIN et al. "BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding". In : *Proceedings of NAACL-HLT*. 2019, p. 4171-4186. URL : <https://aclanthology.org/N19-1423/>.
- [23] Nils REIMERS et Iryna GUREVYCH. "Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks". In : *Proceedings of EMNLP-IJCNLP*. 2019, p. 3982-3992. URL : <https://aclanthology.org/D19-1410/>.

- [24] Tom B. BROWN et al. "Language Models are Few-Shot Learners". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 1877-1901. URL : <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- [25] Colin RAFFEL et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In : *Journal of Machine Learning Research* 21.140 (2020), p. 1-67. URL : <https://www.jmlr.org/papers/v21/20-074.html>.
- [26] Long OUYANG et al. "Training Language Models to Follow Instructions with Human Feedback". In : *Advances in Neural Information Processing Systems* 35 (2022), p. 27730-27744. URL : https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.
- [27] Nandan THAKUR et al. "BEIR : A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models". In : *Advances in Neural Information Processing Systems*. T. 34. 2021, p. 28974-28989. URL : <https://arxiv.org/abs/2104.08663>.
- [28] Niklas MUENNIGHOFF et al. "MTEB : Massive Text Embedding Benchmark". In : *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2023, p. 2014-2037. URL : <https://aclanthology.org/2023.eacl-main.148/>.
- [29] Yu A. MALKOV et D. A. YASHUNIN. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), p. 824-836. DOI : [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [30] Jeff JOHNSON, Matthijs DOUZE et Hervé JÉGOU. "Billion-scale Similarity Search with GPUs". In : *IEEE Transactions on Big Data* 7.3 (2019), p. 535-547. DOI : [10.1109/TBDATA.2019.2921572](https://doi.org/10.1109/TBDATA.2019.2921572).
- [31] Thomas N. KIPF et Max WELLING. "Semi-Supervised Classification with Graph Convolutional Networks". In : *International Conference on Learning Representations*. 2017.
- [32] Petar VELIČKOVIĆ et al. "Graph Attention Networks". In : *International Conference on Learning Representations*. 2018.
- [33] Xiangnan HE et al. "LightGCN : Simplifying and Powering Graph Convolution Network for Recommendation". In : *Proceedings of SIGIR*. 2020, p. 639-648.
- [34] Yunfan GAO et al. *Retrieval-Augmented Generation for Large Language Models : A Survey*. 2023. arXiv : [2312.10997](https://arxiv.org/abs/2312.10997) [cs.CL]. URL : <https://arxiv.org/abs/2312.10997>.

- [35] Shunyu YAO et al. "ReAct : Synergizing Reasoning and Acting in Language Models". In : *International Conference on Learning Representations*. 2023. URL : https://openreview.net/forum?id=WE_vluYUL-X.
- [36] Akari ASAI et al. "Self-RAG : Learning to Retrieve, Generate, and Critique through Self-Reflection". In : *International Conference on Learning Representations*. 2024. URL : <https://openreview.net/forum?id=hSyW5go0v8>.
- [37] Shahul Es et al. "RAGAS : Automated Evaluation of Retrieval Augmented Generation". In : *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics : System Demonstrations*. 2024, p. 150-158. URL : <https://aclanthology.org/2024.eacl-demo.16/>.
- [38] Yang LIU et al. "G-Eval : NLG Evaluation using GPT-4 with Better Human Alignment". In : *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, p. 2511-2522. URL : <https://aclanthology.org/2023.emnlp-main.153/>.
- [39] LANGCHAIN. *LangGraph Documentation*. Documentation officielle du framework LangGraph. 2026. URL : <https://langchain-ai.github.io/langgraph/> (visit  le 06/06/2026).
- [40] Takuya AKIBA et al. "Optuna : A Next-generation Hyperparameter Optimization Framework". In : *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, p. 2623-2631. DOI : [10.1145/3292500.3330701](https://doi.org/10.1145/3292500.3330701).
- [41] William L. HAMILTON. *Graph Representation Learning*. Morgan & Claypool Publishers, 2020. DOI : [10.2200/S01045ED1V01Y202009AIM046](https://doi.org/10.2200/S01045ED1V01Y202009AIM046).

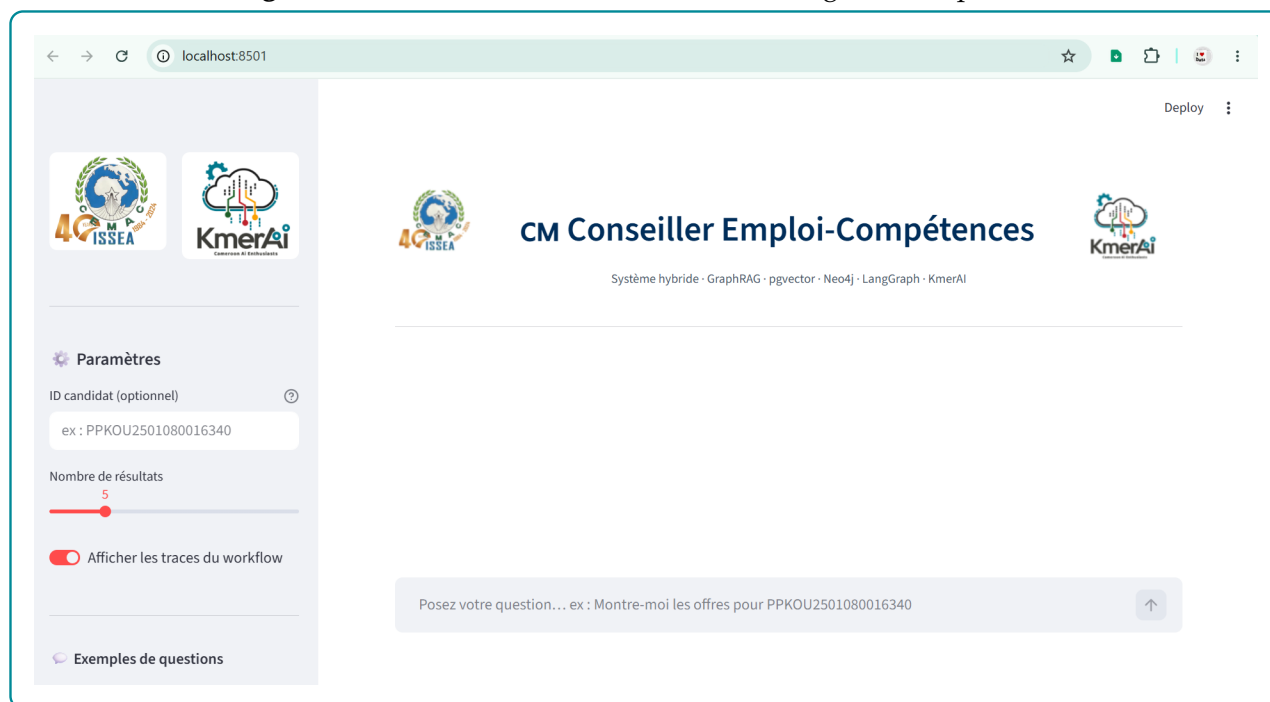
Annexes

Figure A.1 – Architecture méthodologique du workflow agentique de recommandation



Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Figure A.2 – Interface Streamlit du chatbot Agentic GraphRAG



Source : Nos travaux, capture locale de l'interface `chatbot_app.py`.

Table A.1 – Labels de nœuds et relations structurantes du graphe

Bloc	Labels de nœuds	Relations principales et interprétation
Données d'emploi	OffreEmploi, Candidat, Employeur, Secteur, Localisation	PUBLIEE_PAR relie une offre à son employeur; DANS_SECTEUR la rattache à un secteur; LOCALISEE_A indique la ville ou zone d'exercice; A_NIVEAU relie le candidat à son niveau de formation.
Compétences et métiers	Compétence, Métier, GroupeCompétences, GroupeISCO	NECESSITE relie un métier aux compétences attendues; PARTIE_DE rattache une compétence à son groupe; CLASSIFIE_DANS relie un métier à un groupe ISCO; PLUS_LARGE_QUE représente les relations hiérarchiques entre compétences.
Référentiel MEPC	GrandGroupeMEPC, SousGroupeMEPC, GroupeBaseMEPC	CONTIENT représente la hiérarchie interne de la nomenclature; ALIGNE_AVEC rapproche les groupes MEPC des groupes ISCO; CORRESPOND_MEPC rattache un métier ESCO à un groupe de base MEPC.
Référentiel NCF	NiveauFormationNCF, GrandDomaineNCF, DomaineSpécialiséNCF, DomaineDétailléNCF	CONTIENT représente la hiérarchie formation; REQUIERT_NIVEAU_NCF relie une offre au niveau demandé; A_FORMATION rattache un candidat à son domaine de formation; PREPARE_POUR relie un domaine de formation aux métiers visés.
Recommandation	OffreEmploi, Candidat, Compétence, Métier, DomaineDétailléNCF	Les chemins candidat–compétence–offre permettent d'identifier les compétences acquises et manquantes; les chemins offre–niveau–formation servent à contrôler la compatibilité de qualification; les chemins formation–métier soutiennent la proposition de montée en compétences.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.2 – Volume d’entités indexées dans la base vectorielle pgvector

Type d’entité	Nombre	Source principale
OFFRE_EMPLOI	15 722	Scraping KmerAI
COMPETENCE	13 939	Référentiel ESCO v1.1
METIER	3 039	Référentiel ESCO v1.1
CANDIDAT	1 105	Base locale de profils demandeurs
GROUPE_BASE_MEPC	209	INS Cameroun - MEPC 2013
DOMAINE_DETAILLE_NCF	201	INS Cameroun - NCF 2017
Total	34 215	

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.3 – Pipeline général implémenté : étapes, entrées et sorties

Ordre	Étape	Entrées principales	Sorties concrètes
1	ETL	Données brutes d’offres, candidats et référentiels	Fichiers normalisés, champs <code>text_to_embed</code> , paires <code>pairs_train/val/test.jsonl</code> .
2	Fine-tuning	Paires offre–description et modèle <code>all-MiniLM-L6-v2</code>	Modèle sauvegardé dans <code>models/st_finetuned/final</code> et métriques d’entraînement.
3	Indexation vectorielle	Entités nettoyées et encodeur fine-tuné	Table <code>embeddings</code> , table <code>doc_chunks</code> , index HNSW et index textuels PostgreSQL.
4	Graphe de connaissances	Offres, candidats, compétences, métiers, niveaux et référentiels	Noeuds Neo4j, relations métier-compétence-offre-candidat, contraintes et chemins explicatifs.
5	GraphRAG hybride	Résultats pgvector, chemins Neo4j et règles métier	Contexte candidat–offre, score hybride, analyse de <i>skill gap</i> , critic et Text2Cypher.
6	Orchestration LangGraph	Question utilisateur et registre d’outils	État partagé, routage, exécution des outils, génération finale et traces du workflow.
7	Exposition applicative	Workflow agentique et services locaux	API FastAPI, CLI de test et interface Streamlit.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.4 – Correspondance entre objectifs spécifiques et choix méthodologiques

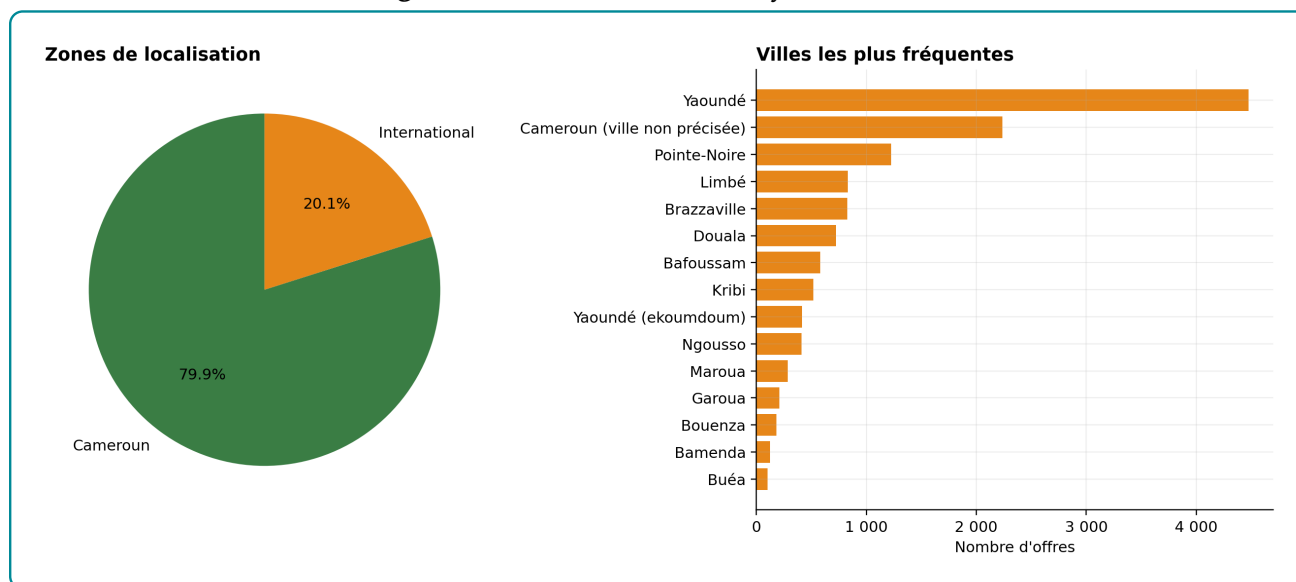
Objectif spécifique	Choix méthodologique retenu
Normaliser les offres, candidats et référentiels	Construction d'une couche de données standardisée : identifiants stables, champs nettoyés, niveaux de formation, secteurs, compétences et textes d'embedding.
Adapter la recherche sémantique au domaine emploi-compétences	Fine-tuning contrastif d'un modèle SentenceTransformer, puis indexation des embeddings dans PostgreSQL avec pgvector pour le retrieval top-K [23, 2].
Représenter les relations entre métiers, compétences et profils	Construction d'un graphe de connaissances Neo4j reliant candidats, offres, compétences, métiers, secteurs, localisations et référentiels [3, 41].
Produire une recommandation explicable	Score hybride combinant similarité vectorielle, couverture de compétences, compatibilité de niveau et alignement sectoriel, complété par une analyse de <i>skill gap</i> .
Orchestrer la réponse utilisateur	Workflow LangGraph routant la requête vers les outils adaptés : pgvector, Neo4j, recherche documentaire, Text2Cypher, critic et génération finale.
Evaluer la qualité du système	Protocole séparant l'évaluation du retrieval, du graphe, du classement et de la réponse générée.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.5 – Volumes réellement exploités par les modules de recommandation

Objet exploité	Volume	Utilité pour le système
Offres normalisées	13 957	Base principale à recommander, indexée dans pgvector et reliée au graphe.
Candidats normalisés	41 298	Vivier à partir duquel sont calculés les rapprochements offre–profil.
Compétences indexées	13 939	Signal de matching, de filtrage et de justification du <i>skill gap</i> .
Secteurs nettoyés	1 039	Signal de contexte pour limiter les rapprochements absurdes entre domaines trop éloignés.
Métiers indexés	3 039	Niveau d’agrégation utile pour stabiliser les recommandations au-delà des intitulés d’offres.
Groupes métiers MEPC	209	Référentiel externe pour structurer les proximités métier.
Domaines détaillés NCF	201	Référentiel de formation utilisé pour contextualiser les niveaux et domaines.
Localisations représentées dans Neo4j	312	Signal géographique pour filtrage souple et explication des résultats.

Source : Nos travaux, diagnostics PostgreSQL

Figure A.3 – Localisation nettoyée des offres

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.6 – Synthèse des traitements de nettoyage utiles au moteur de recommandation

Traitement	Décision implémentée	Effet attendu
Casse et formats	Harmonisation des libellés pour réduire les variantes d'écriture.	Moins de fragmentation dans les compétences, secteurs, villes et sources.
Doublons	Validation uniquement si toute la ligne est identique sur toutes les variables.	Suppression prudente des répétitions exactes, sans effacer des offres proches mais distinctes.
Localisation	Séparation Cameroun, international et non précisée; conservation des localités fines disponibles.	Filtrage géographique possible sans perte des offres incomplètes.
Champs vides	Modalités explicites lorsque le champ reste exploitable comme absence d'information.	Le système peut pénaliser ou contextualiser le manque d'information au lieu d'ignorer l'offre.
Descriptions faibles	Enrichissement de la représentation par métadonnées et compétences.	Robustesse accrue pour les sources où le détail de l'offre est absent ou court.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.7 – Comparaison directe entre le baseline et le modèle fine-tuné

Métrique test	Baseline	Fine-tuné	Gain
NDCG@10	0,1681	0,6232	+0,4552
MRR@10	0,1410	0,5874	+0,4463
Recall@1	0,0986	0,5268	+0,4281
Recall@5	0,1983	0,6632	+0,4648
Recall@10	0,2560	0,7398	+0,4837
Precision@1	0,0986	0,5268	+0,4281

Source : Nos travaux, traitements Python et SentenceTransformer

Table A.8 – Diagnostic statistique du chunking documentaire

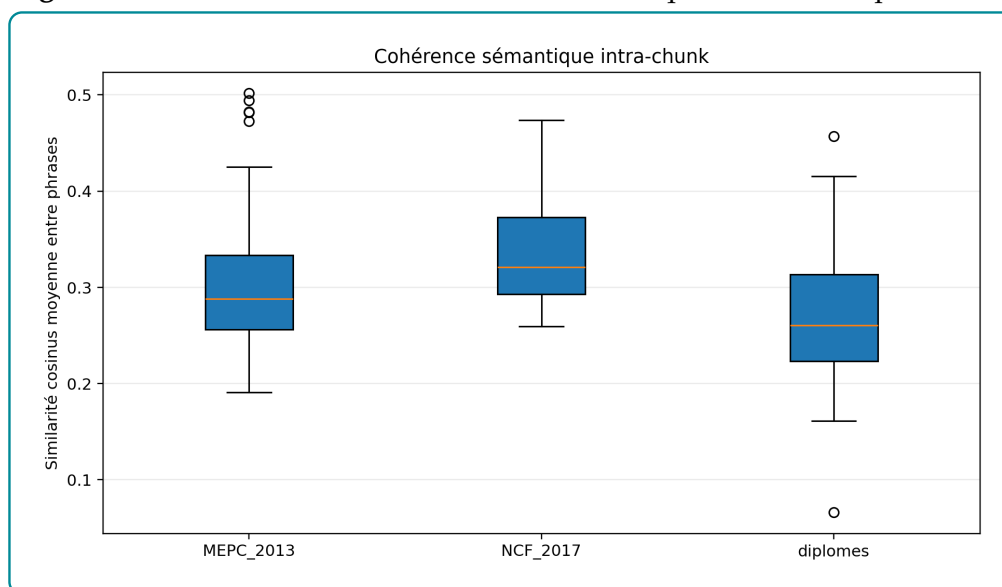
Indicateur	Valeur observée
Documents sources indexés	3
Chunks documentaires indexés	142
Stratégie réellement utilisée	100 % structurelle
Taille moyenne	187,1 tokens
Taille médiane	116,0 tokens
Écart-type	169,6 tokens
Minimum / maximum	6 / 492 tokens
Coefficient de variation	0,91
Chunks avec embedding	142 / 142
Chunks synchronisés avec Neo4j	142 / 142

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.9 – Couverture documentaire et duplication des chunks

Source	Couverture approximative	Chunks
NCF 2017	25,9 %	38
MEPC 2013	23,6 %	61
Référentiel des diplômes	79,3 %	43
Taux de duplication cosinus $> 0,95$	0,0 %	142
Similarité maximale hors diagonale	0,921	–

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

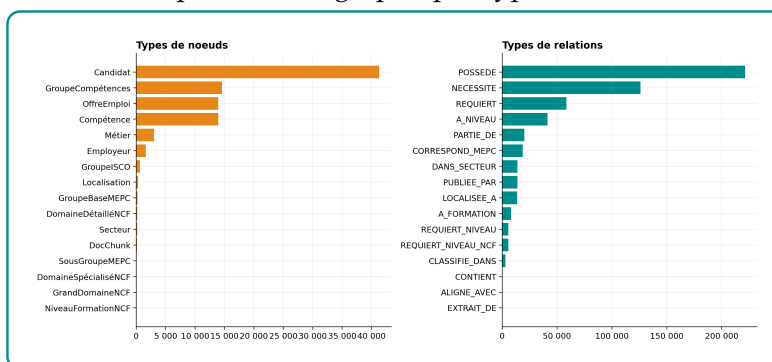
Figure A.4 – Distribution de la cohérence sémantique intra-chunk par source

Source : Nos travaux, mesure Python de la similarité cosinus moyenne entre les phrases de chaque chunk.

Table A.10 – Cohérence sémantique intra-chunk

Source	Moyenne	Médiane	Intervalle observé
MEPC 2013	0,307	0,288	0,191–0,502
NCF 2017	0,336	0,321	0,259–0,474
Référentiel des diplômes	0,268	0,260	0,066–0,457

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Figure A.5 – Composition du graphe par types de noeuds et relations

Source : Nos travaux, diagnostics Neo4j.

Table A.11 – Hétérogénéité des degrés par famille de noeuds

Label	Moyenne	Médiane	p90	Max
NiveauFormationNCF	5 847,88	5 924,5	10 056,6	10 261
GroupeBaseMEPC	95,11	74	200	275
Secteur	72,69	5,5	151,6	1 548
Métier	47,61	43	76	185
Localisation	43,96	1	8	4 467
Compétence	30,04	4	27	6 787
OffreEmploi	6,57	8	9	9
Candidat	6,36	7	7	7

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

Table A.12 – Résumé du déplacement de rang après reranking Neo4j

Indicateur	Valeur
Couples candidat–offre évalués	690
Candidats distincts	69
Rang moyen avant reranking pgvector	9,51
Rang moyen après reranking Neo4j/Optuna	5,50
Déplacement moyen de rang	+4,01
Déplacement médian	+1,00
Offres remontées	59,86 %
Offres rétrogradées	13,77 %
Offres inchangées	26,38 %

Source : Nos travaux, traitements Python de nettoyage et de diagnostic

Table des matières

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	v
Liste des tableaux	vi
Liste des figures	vii
Avant-propos	viii
Résumé	ix
Abstract	x
Introduction générale	1
I Cadre théorique et conceptuel	6
1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation	7
1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG	7
1.2 Fondements économiques et problématique d'appariement	9
1.2.1 Marché du travail, recherche d'emploi et frictions	9
1.2.2 Capital humain, tâches et inadéquation des compétences	10
1.3 Systèmes de recommandation et person-job fit	10
1.3.1 Approches par contenu, collaboratives et hybrides	10
1.3.2 Spécificité du person-job fit	11
1.4 LLMs, embeddings et graphes de connaissances	12
1.4.1 Grands modèles de langage et génération contrôlée	12
1.4.2 Embeddings, recherche sémantique et bases vectorielles	13
1.4.3 Graphes de connaissances et recommandation relationnelle	13
1.5 RAG, GraphRAG et orchestration agentique	14
1.5.1 De la RAG au GraphRAG	14
1.5.2 Agentic RAG, critic et traçabilité	15

1.5.3	Évaluer la performance et la factualité	16
1.6	Positionnement de l'étude	18
2	Sources de données et approche méthodologique	19
2.1	Cadre méthodologique et sources de données	19
2.1.1	Démarche d'ingénierie appliquée retenue	19
2.1.2	Sources de données et leur rôle dans le système	20
2.1.3	Préparation et normalisation des données	21
2.1.4	Alignement des référentiels métiers et formations	21
2.2	Représentation sémantique et graphe de connaissances	21
2.2.1	Choix du modèle d'embeddings et fine-tuning	21
2.2.2	Indexation vectorielle dans PostgreSQL et pgvector	22
2.2.3	Construction du graphe de connaissances sous Neo4j	22
2.2.4	Score hybride et logique de skill gap	22
2.2.5	Construction du proxy de pertinence pour l'évaluation	22
2.3	Orchestration agentique et génération contrôlée	24
2.3.1	Architecture Agentic RAG et orchestration LangGraph	24
2.3.2	Text2Cypher et rôle du LLM	24
2.3.3	Modes d'accès et traçabilité des décisions	24
2.3.4	Protocole d'évaluation prévu du système	24
II	Présentation des résultats	25
3	Construction opérationnelle du système hybride emploi-compétences	26
3.1	Données exploitables et contraintes de qualité du système	26
3.1.1	Volumes disponibles après nettoyage	26
3.1.2	Qualité des variables utiles au matching emploi-compétences	27
3.1.2.1	Localisation exploitable pour filtrer sans exclure	27
3.1.2.2	Compétences et descriptions : signal utile mais hétérogène	27
3.1.3	Nettoyage réalisé et limites conservées	27
3.2	Adaptation du modèle d'embeddings au domaine emploi-compétences	28
3.2.1	Encodeur retenu et paires contrastives	28
3.2.2	Fine-tuning : contraintes de calcul et preuve de gain	29
3.2.2.1	Paramétrage contraint et dynamique d'apprentissage	30
3.2.2.2	Preuve de gain sur le ranking sémantique	31
3.2.3	Structure de l'espace vectoriel après fine-tuning	32
3.3	Mémoire vectorielle pgvector pour le retrieval	33
3.3.1	Schéma physique, tables et volumes indexés	33

3.3.2	Référentiels découpés en chunks interrogeables	34
3.4	Neo4j : mémoire relationnelle du matching	35
3.4.1	Schéma, noeuds et relations mobilisés par le matching	35
3.4.2	Structure du graphe et dépendance aux hubs	37
3.5	Score hybride et reranking des recommandations	39
3.5.1	Formalisation du score hybride	39
3.5.2	Effet du reranking hybride	40
3.6	Orchestration du système par LangGraph	40
3.6.1	Analyse de la requête et choix des outils	41
3.6.2	Exécution des outils et construction du contexte	41
3.6.3	Génération contrôlée et boucle de révision	42
4	Évaluation de la performance du système	43
4.1	Protocole expérimental et métriques retenues	43
4.2	Retrieval sémantique et apport mesuré du graphe	44
4.2.1	Modèle d’embeddings spécialisé	44
4.2.2	Ablation pgvector seul contre pgvector enrichi par Neo4j	45
4.3	Validation fonctionnelle du GraphRAG	46
4.3.1	Requêtes relationnelles métier, compétence et offre	48
4.3.2	Skill gap, niveau NCF et comparaison GraphRAG	48
4.4	Reranking hybride et calibration du score	49
4.4.1	Déplacements de rang après reranking par le graphe	49
4.4.2	Pondérations optimales avec Optuna	50
4.5	Contrôle de génération par critic	52
4.5.1	Calibration contrastive du seuil de fidélité	52
4.5.2	Interprétation des décisions accept/revise	53
	Conclusion générale	55
	Bibliographie	II
A	Annexes	VI